

```

case DXGI_FORMAT_BC2_TYPELESS:
case DXGI_FORMAT_BC2_UNORM:
case DXGI_FORMAT_BC2_UNORM_SRGB:
case DXGI_FORMAT_BC3_TYPELESS:
case DXGI_FORMAT_BC3_UNORM:
case DXGI_FORMAT_BC3_UNORM_SRGB:
case DXGI_FORMAT_BC5_TYPELESS:
case DXGI_FORMAT_BC5_UNORM:
case DXGI_FORMAT_BC5_SNORM:
case DXGI_FORMAT_BC6H_TYPELESS:
case DXGI_FORMAT_BC6H_UF16:
case DXGI_FORMAT_BC6H_SF16:
case DXGI_FORMAT_BC7_TYPELESS:
case DXGI_FORMAT_BC7_UNORM:
case DXGI_FORMAT_BC7_UNORM_SRGB:
    bc = true;
    bcnBytesPerBlock = 16;
    break;

case DXGI_FORMAT_R8G8_B8G8_UNORM:
case DXGI_FORMAT_G8R8_G8B8_UNORM:
    packed = true;
    break;
}

if (bc)
{
    size_t numBlockWide = 0;
    if (width > 0)
    {
        numBlockWide = std::max<size_t>(1, (width + 3) / 4);
    }
    size_t numBlockHigh = 0;
    if (height > 0)
    {
        numBlockHigh = std::max<size_t>(1, (height + 3) / 4);
    }
    rowBytes = numBlockWide * bcnBytesPerBlock;
    numRows = numBlockHigh;
}
else if (packed)
{
    rowBytes = ((width + 1) >> 1) * 4;
    numRows = height;
}
else
{
    size_t bpp = BitsPerPixel(fmt);
    rowBytes = (width * bpp + 7) / 8; // Round up to the nearest byte.
    numRows = height;
}

numBytes = rowBytes * numRows;
if (outNumBytes)

```

```

    {
        *outNumBytes = numBytes;
    }
    if (outRowBytes)
    {
        *outRowBytes = rowBytes;
    }
    if (outNumRows)
    {
        *outNumRows = numRows;
    }
}

//-----
-----
#define ISBITMASK(r, g, b, a) (ddpf.RBitMask == r && ddpf.GBitMask == g &&
ddpf.BBitMask == b && ddpf.ABitMask == a)

static DXGI_FORMAT GetDXGIFormat(const DDS_PIXELFORMAT& ddpf)
{
    if (ddpf.flags & DDS_RGB)
    {
        // Note that sRGB formats are written using the "DX10" extended
header.

        switch (ddpf.RGBBitCount)
        {
            case 32:
                if (ISBITMASK(0x000000ff, 0x0000ff00, 0x00ff0000, 0xff000000))
                {
                    return DXGI_FORMAT_R8G8B8A8_UNORM;
                }

                if (ISBITMASK(0x00ff0000, 0x0000ff00, 0x000000ff, 0xff000000))
                {
                    return DXGI_FORMAT_B8G8R8A8_UNORM;
                }

                if (ISBITMASK(0x00ff0000, 0x0000ff00, 0x000000ff, 0x00000000))
                {
                    return DXGI_FORMAT_B8G8R8X8_UNORM;
                }

                // No DXGI format maps to ISBITMASK(0x000000ff, 0x0000ff00,
0x00ff0000, 0x00000000) aka D3DFMT_X8B8G8R8

                // Note that many common DDS reader/writers (including D3DX)
swap the
                // the RED/BLE masks for 10:10:10:2 formats. We assume
                // below that the 'backwards' header mask is being used since it
is most
                // likely written by D3DX. The more robust solution is to use
the 'DX10'
                // header extension and specify the

```

DXGI_FORMAT_R10G10B10A2_UNORM format directly

```
        // For 'correct' writers, this should be 0x000003ff, 0x000ffc00,
0x3ff00000 for RGB data
        if (ISBITMASK(0x3ff00000, 0x000ffc00, 0x000003ff, 0xc0000000))
        {
            return DXGI_FORMAT_R10G10B10A2_UNORM;
        }

        // No DXGI format maps to ISBITMASK(0x000003ff, 0x000ffc00,
0x3ff00000, 0xc0000000) aka D3DFMT_A2R10G10B10

        if (ISBITMASK(0x0000ffff, 0xffff0000, 0x00000000, 0x00000000))
        {
            return DXGI_FORMAT_R16G16_UNORM;
        }

        if (ISBITMASK(0xffffffff, 0x00000000, 0x00000000, 0x00000000))
        {
            // Only 32-bit color channel format in D3D9 was R32F.
            return DXGI_FORMAT_R32_FLOAT; // D3DX writes this out as a
FourCC of 114.
        }
        break;

    case 24:
        // No 24bpp DXGI formats aka D3DFMT_R8G8B8
        break;

    case 16:
        if (ISBITMASK(0x7c00, 0x03e0, 0x001f, 0x8000))
        {
            return DXGI_FORMAT_B5G5R5A1_UNORM;
        }
        if (ISBITMASK(0xf800, 0x07e0, 0x001f, 0x0000))
        {
            return DXGI_FORMAT_B5G6R5_UNORM;
        }

        // No DXGI format maps to ISBITMASK(0x7c00, 0x03e0, 0x001f,
0x0000) aka D3DFMT_X1R5G5B5.
        if (ISBITMASK(0x0f00, 0x00f0, 0x000f, 0xf000))
        {
            return DXGI_FORMAT_B4G4R4A4_UNORM;
        }

        // No DXGI format maps to ISBITMASK(0x0f00, 0x00f0, 0x000f,
0x0000) aka D3DFMT_X4R4G4B4.

        // No 3:3:2, 3:3:2:8, or paletted DXGI formats aka
D3DFMT_A8R3G3B2, D3DFMT_R3G3B2, D3DFMT_P8, D3DFMT_A8P8, etc.
        break;
    }
}
else if (ddpf.flags & DDS_LUMINANCE)
```

```

{
    if (8 == ddpf.RGBBitCount)
    {
        if (ISBITMASK(0x000000ff, 0x00000000, 0x00000000, 0x00000000))
        {
            return DXGI_FORMAT_R8_UNORM; // D3DX10/11 writes this out as
DX10 extension
        }

        // No DXGI format maps to ISBITMASK(0x0f, 0x00, 0x00, 0xf0) aka
D3DFMT_A4L4.
    }

    if (16 == ddpf.RGBBitCount)
    {
        if (ISBITMASK(0x0000ffff, 0x00000000, 0x00000000, 0x00000000))
        {
            return DXGI_FORMAT_R16_UNORM; // D3DX10/11 writes this out
as DX10 extension.
        }
        if (ISBITMASK(0x000000ff, 0x00000000, 0x00000000, 0x0000ff00))
        {
            return DXGI_FORMAT_R8G8_UNORM; // D3DX10/11 writes this out
as DX10 extension.
        }
    }
}
else if (ddpf.flags & DDS_ALPHA)
{
    if (8 == ddpf.RGBBitCount)
    {
        return DXGI_FORMAT_A8_UNORM;
    }
}
else if (ddpf.flags & DDS_FOURCC)
{
    if (MAKEFOURCC('D', 'X', 'T', '1') == ddpf.fourCC)
    {
        return DXGI_FORMAT_BC1_UNORM;
    }
    if (MAKEFOURCC('D', 'X', 'T', '3') == ddpf.fourCC)
    {
        return DXGI_FORMAT_BC2_UNORM;
    }
    if (MAKEFOURCC('D', 'X', 'T', '5') == ddpf.fourCC)
    {
        return DXGI_FORMAT_BC3_UNORM;
    }

    // While pre-multiplied alpha isn't directly supported by the DXGI
formats,
    // they are basically the same as these BC formats so they can be
mapped
    if (MAKEFOURCC('D', 'X', 'T', '2') == ddpf.fourCC)
    {

```

```

        return DXGI_FORMAT_BC2_UNORM;
    }
    if (MAKEFOURCC('D', 'X', 'T', '4') == ddpf.fourCC)
    {
        return DXGI_FORMAT_BC3_UNORM;
    }

    if (MAKEFOURCC('A', 'T', 'I', '1') == ddpf.fourCC)
    {
        return DXGI_FORMAT_BC4_UNORM;
    }
    if (MAKEFOURCC('B', 'C', '4', 'U') == ddpf.fourCC)
    {
        return DXGI_FORMAT_BC4_UNORM;
    }
    if (MAKEFOURCC('B', 'C', '4', 'S') == ddpf.fourCC)
    {
        return DXGI_FORMAT_BC4_SNORM;
    }

    if (MAKEFOURCC('A', 'T', 'I', '2') == ddpf.fourCC)
    {
        return DXGI_FORMAT_BC5_UNORM;
    }
    if (MAKEFOURCC('B', 'C', '5', 'U') == ddpf.fourCC)
    {
        return DXGI_FORMAT_BC5_UNORM;
    }
    if (MAKEFOURCC('B', 'C', '5', 'S') == ddpf.fourCC)
    {
        return DXGI_FORMAT_BC5_SNORM;
    }

    // BC6H and BC7 are written using the "DX10" extended header

    if (MAKEFOURCC('R', 'G', 'B', 'G') == ddpf.fourCC)
    {
        return DXGI_FORMAT_R8G8_B8G8_UNORM;
    }
    if (MAKEFOURCC('G', 'R', 'G', 'B') == ddpf.fourCC)
    {
        return DXGI_FORMAT_G8R8_G8B8_UNORM;
    }

    // Check for D3DFORMAT enums being set here.
    switch (ddpf.fourCC)
    {
    case 36: // D3DFMT_A16B16G16R16
        return DXGI_FORMAT_R16G16B16A16_UNORM;

    case 110: // D3DFMT_Q16W16V16U16
        return DXGI_FORMAT_R16G16B16A16_SNORM;

    case 111: // D3DFMT_R16F
        return DXGI_FORMAT_R16_FLOAT;
    }

```

```

        case 112: // D3DFMT_G16R16F
            return DXGI_FORMAT_R16G16_FLOAT;

        case 113: // D3DFMT_A16B16G16R16F
            return DXGI_FORMAT_R16G16B16A16_FLOAT;

        case 114: // D3DFMT_R32F
            return DXGI_FORMAT_R32_FLOAT;

        case 115: // D3DFMT_G32R32F
            return DXGI_FORMAT_R32G32_FLOAT;

        case 116: // D3DFMT_A32B32G32R32F
            return DXGI_FORMAT_R32G32B32A32_FLOAT;
    }
}

return DXGI_FORMAT_UNKNOWN;
}

//-----
static void FillInitData(
    _In_ size_t width,
    _In_ size_t height,
    _In_ size_t depth,
    _In_ size_t mipCount,
    _In_ size_t arraySize,
    _In_ DXGI_FORMAT format,
    _In_ size_t maxsize,
    _In_ size_t bitSize,
    _In_reads_bytes_(bitSize) const byte* bitData,
    _Out_ size_t& twidth,
    _Out_ size_t& theight,
    _Out_ size_t& tdepth,
    _Out_ size_t& skipMip,
    _Out_writes_(mipCount*arraySize) D3D11_SUBRESOURCE_DATA* initData
)
{
    if (!bitData || !initData)
    {
        throw ref new Platform::InvalidArgumentException();
    }

    skipMip = 0;
    twidth = 0;
    theight = 0;
    tdepth = 0;

    size_t NumBytes = 0;
    size_t RowBytes = 0;
    size_t NumRows = 0;
    const byte* pSrcBits = bitData;

```

```

const byte* pEndBits = bitData + bitSize;

size_t index = 0;
for (size_t j = 0; j < arraySize; j++)
{
    size_t w = width;
    size_t h = height;
    size_t d = depth;
    for (size_t i = 0; i < mipCount; i++)
    {
        GetSurfaceInfo(w, h, format, &NumBytes, &RowBytes, &NumRows);

        if ((mipCount <= 1) || !maxsize || (w <= maxsize && h <= maxsize
&& d <= maxsize))
        {
            if (!twidth)
            {
                twidth = w;
                theight = h;
                tdepth = d;
            }

            initData[index].pSysMem = (const void*)pSrcBits;
            initData[index].SysMemPitch = static_cast<UINT>(RowBytes);
            initData[index].SysMemSlicePitch = static_cast<UINT>
(NumBytes);
            ++index;
        }
        else
        {
            ++skipMip;
        }

        if (pSrcBits + (NumBytes*d) > pEndBits)
        {
            throw ref new Platform::OutOfBoundsException();
        }

        pSrcBits += NumBytes * d;

        w = w >> 1;
        h = h >> 1;
        d = d >> 1;
        if (w == 0)
        {
            w = 1;
        }
        if (h == 0)
        {
            h = 1;
        }
        if (d == 0)
        {
            d = 1;
        }
    }
}

```

```

    }
}

if (!index)
{
    throw ref new Platform::FailureException();
}
}

//-----
static HRESULT CreateD3DResources(
    _In_ ID3D11Device* d3dDevice,
    _In_ uint32 resDim,
    _In_ size_t width,
    _In_ size_t height,
    _In_ size_t depth,
    _In_ size_t mipCount,
    _In_ size_t arraySize,
    _In_ DXGI_FORMAT format,
    _In_ bool isCubeMap,
    _In_reads_(mipCount*arraySize) D3D11_SUBRESOURCE_DATA* initData,
    _Out_opt_ ID3D11Resource** texture,
    _Out_opt_ ID3D11ShaderResourceView** textureView
)
{
    if (!d3dDevice || !initData)
    {
        return E_INVALIDARG;
    }

    HRESULT hr = E_FAIL;

    switch (resDim)
    {
        case D3D11_RESOURCE_DIMENSION_TEXTURE1D:
        {
            D3D11_TEXTURE1D_DESC desc;
            desc.Width = static_cast<UINT>(width);
            desc.MipLevels = static_cast<UINT>(mipCount);
            desc.ArraySize = static_cast<UINT>(arraySize);
            desc.Format = format;
            desc.Usage = D3D11_USAGE_DEFAULT;
            desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;
            desc.CPUAccessFlags = 0;
            desc.MiscFlags = 0;

            ID3D11Texture1D* tex = nullptr;
            hr = d3dDevice->CreateTexture1D(&desc, initData, &tex);

            if (SUCCEEDED(hr) && tex != 0)
            {
                if (textureView != 0)
                {

```



```

        D3D11_SHADER_RESOURCE_VIEW_DESC SRVDesc;
        memset(&SRVDesc, 0, sizeof(SRVDesc));
        SRVDesc.Format = format;

        if (arraySize > 1)
        {
            SRVDesc.ViewDimension =
D3D_SRV_DIMENSION_TEXTURE1DARRAY;
            SRVDesc.Texture1DArray.MipLevels =
desc.MipLevels;
            SRVDesc.Texture1DArray.ArraySize =
static_cast<UINT>(arraySize);
        }
        else
        {
            SRVDesc.ViewDimension =
D3D_SRV_DIMENSION_TEXTURE1D;
            SRVDesc.Texture1D.MipLevels = desc.MipLevels;
        }

        hr = d3dDevice->CreateShaderResourceView(tex,
&SRVDesc, textureView);

        if (FAILED(hr))
        {
            tex->Release();
            return hr;
        }
    }

    if (texture != 0)
    {
        *texture = tex;
    }
    else
    {
        tex->Release();
    }
}
break;

case D3D11_RESOURCE_DIMENSION_TEXTURE2D:
{
    D3D11_TEXTURE2D_DESC desc;
    desc.Width = static_cast<UINT>(width);
    desc.Height = static_cast<UINT>(height);
    desc.MipLevels = static_cast<UINT>(mipCount);
    desc.ArraySize = static_cast<UINT>(arraySize);
    desc.Format = format;
    desc.SampleDesc.Count = 1;
    desc.SampleDesc.Quality = 0;
    desc.Usage = D3D11_USAGE_DEFAULT;
    desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;
    desc.CPUAccessFlags = 0;

```

```

        desc.MiscFlags = (isCubeMap) ?
D3D11_RESOURCE_MISC_TEXTURECUBE : 0;

ID3D11Texture2D* tex = nullptr;
hr = d3dDevice->CreateTexture2D(&desc, initData, &tex);

if (SUCCEEDED(hr) && tex != 0)
{
    if (textureView != 0)
    {
        D3D11_SHADER_RESOURCE_VIEW_DESC SRVDesc;
        memset(&SRVDesc, 0, sizeof(SRVDesc));
        SRVDesc.Format = format;

        if (isCubeMap)
        {
            if (arraySize > 6)
            {
                SRVDesc.ViewDimension =
D3D_SRV_DIMENSION_TEXTURECUBEARRAY;
                SRVDesc.TextureCubeArray.MipLevels =
desc.MipLevels;

                // Earlier, we set arraySize to (NumCubes *
6).
                SRVDesc.TextureCubeArray.NumCubes =
static_cast<UINT>(arraySize / 6);
            }
            else
            {
                SRVDesc.ViewDimension =
D3D_SRV_DIMENSION_TEXTURECUBE;
                SRVDesc.TextureCube.MipLevels =
desc.MipLevels;
            }
        }
        else if (arraySize > 1)
        {
            SRVDesc.ViewDimension =
D3D_SRV_DIMENSION_TEXTURE2DARRAY;
            SRVDesc.Texture2DArray.MipLevels =
desc.MipLevels;
            SRVDesc.Texture2DArray.ArraySize =
static_cast<UINT>(arraySize);
        }
        else
        {
            SRVDesc.ViewDimension =
D3D_SRV_DIMENSION_TEXTURE2D;
            SRVDesc.Texture2D.MipLevels = desc.MipLevels;
        }

        hr = d3dDevice->CreateShaderResourceView(tex,
&SRVDesc, textureView);

```

```

        if (FAILED(hr))
        {
            tex->Release();
            return hr;
        }
    }

    if (texture != 0)
    {
        *texture = tex;
    }
    else
    {
        tex->Release();
    }
}

break;

case D3D11_RESOURCE_DIMENSION_TEXTURE3D:
{
    D3D11_TEXTURE3D_DESC desc;
    desc.Width = static_cast<UINT>(width);
    desc.Height = static_cast<UINT>(height);
    desc.Depth = static_cast<UINT>(depth);
    desc.MipLevels = static_cast<UINT>(mipCount);
    desc.Format = format;
    desc.Usage = D3D11_USAGE_DEFAULT;
    desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;
    desc.CPUAccessFlags = 0;
    desc.MiscFlags = 0;

    ID3D11Texture3D* tex = nullptr;
    hr = d3dDevice->CreateTexture3D(&desc, initData, &tex);

    if (SUCCEEDED(hr) && tex != 0)
    {
        if (textureView != 0)
        {
            D3D11_SHADER_RESOURCE_VIEW_DESC SRVDesc;
            memset(&SRVDesc, 0, sizeof(SRVDesc));
            SRVDesc.Format = format;
            SRVDesc.ViewDimension = D3D_SRV_DIMENSION_TEXTURE3D;
            SRVDesc.Texture3D.MipLevels = desc.MipLevels;

            hr = d3dDevice->CreateShaderResourceView(tex,
&SRVDesc, textureView);

            if (FAILED(hr))
            {
                tex->Release();
                return hr;
            }
        }
    }
}

```

```

        if (texture != 0)
        {
            *texture = tex;
        }
        else
        {
            tex->Release();
        }
    }
}
break;
}

return hr;
}

//-----
-----
static void CreateTextureFromDDS(
    _In_ ID3D11Device* d3dDevice,
    _In_ const DDS_HEADER* header,
    _In_reads_bytes_(bitSize) const byte* bitData,
    _In_ size_t bitSize,
    _Out_opt_ ID3D11Resource** texture,
    _Out_opt_ ID3D11ShaderResourceView** textureView,
    _In_ size_t maxsize
)
{
    HRESULT hr = S_OK;

    size_t width = header->width;
    size_t height = header->height;
    size_t depth = header->depth;

    uint32 resDim = D3D11_RESOURCE_DIMENSION_UNKNOWN;
    size_t arraySize = 1;
    DXGI_FORMAT format = DXGI_FORMAT_UNKNOWN;
    bool isCubeMap = false;

    size_t mipCount = header->mipMapCount;
    if (0 == mipCount)
    {
        mipCount = 1;
    }

    if ((header->ddspf.flags & DDS_FOURCC) &&
        (MAKEFOURCC('D', 'X', '1', '0') == header->ddspf.fourCC))
    {
        const DDS_HEADER_DXT10* d3d10ext = reinterpret_cast<const
DDS_HEADER_DXT10*>((const char*)header + sizeof(DDS_HEADER));

        arraySize = d3d10ext->arraySize;
        if (arraySize == 0)
        {

```

```

        throw ref new Platform::FailureException();
    }

    if (BitsPerPixel(d3d10ext->dxgiFormat) == 0)
    {
        throw ref new Platform::FailureException();
    }

    format = d3d10ext->dxgiFormat;

    switch (d3d10ext->resourceDimension)
    {
    case D3D11_RESOURCE_DIMENSION_TEXTURE1D:
        // D3DX writes 1D textures with a fixed Height of 1.
        if ((header->flags & DDS_HEIGHT) && height != 1)
        {
            throw ref new Platform::FailureException();
        }
        height = depth = 1;
        break;

    case D3D11_RESOURCE_DIMENSION_TEXTURE2D:
        if (d3d10ext->miscFlag & D3D11_RESOURCE_MISC_TEXTURECUBE)
        {
            arraySize *= 6;
            isCubeMap = true;
        }
        depth = 1;
        break;

    case D3D11_RESOURCE_DIMENSION_TEXTURE3D:
        if (!(header->flags & DDS_HEADER_FLAGS_VOLUME))
        {
            throw ref new Platform::FailureException();
        }

        if (arraySize > 1)
        {
            throw ref new Platform::FailureException();
        }
        break;

    default:
        return throw ref new Platform::FailureException();
    }

    resDim = d3d10ext->resourceDimension;
}
else
{
    format = GetDXGIFormat(header->ddspf);

    if (format == DXGI_FORMAT_UNKNOWN)
    {
        return throw ref new Platform::FailureException();
    }
}

```

```

    }

    if (header->flags & DDS_HEADER_FLAGS_VOLUME)
    {
        resDim = D3D11_RESOURCE_DIMENSION_TEXTURE3D;
    }
    else
    {
        if (header->caps2 & DDS_CUBEMAP)
        {
            // We require all six faces to be defined.
            if ((header->caps2 & DDS_CUBEMAP_ALLFACES) !=
DDS_CUBEMAP_ALLFACES)
            {
                return throw ref new Platform::FailureException();
            }

            arraySize = 6;
            isCubeMap = true;
        }

        depth = 1;
        resDim = D3D11_RESOURCE_DIMENSION_TEXTURE2D;

        // Note there's no way for a legacy Direct3D 9 DDS to express a
'1D' texture.
    }

    assert(BitsPerPixel(format) != 0);
}

// Bound sizes (For security purposes, we don't trust DDS file metadata
larger than the D3D 11.x hardware requirements.)
if (mipCount > D3D11_REQ_MIP_LEVELS)
{
    return throw ref new Platform::FailureException();
}

switch (resDim)
{
case D3D11_RESOURCE_DIMENSION_TEXTURE1D:
    if ((arraySize > D3D11_REQ_TEXTURE1D_ARRAY_AXIS_DIMENSION) ||
        (width > D3D11_REQ_TEXTURE1D_U_DIMENSION))
    {
        return throw ref new Platform::FailureException();
    }
    break;

case D3D11_RESOURCE_DIMENSION_TEXTURE2D:
    if (isCubeMap)
    {
        // This is the right bound because we set arraySize to
(NumCubes*6) above.
        if ((arraySize > D3D11_REQ_TEXTURE2D_ARRAY_AXIS_DIMENSION)
||

```

```

        (width > D3D11_REQ_TEXTURECUBE_DIMENSION) ||
        (height > D3D11_REQ_TEXTURECUBE_DIMENSION))
    {
        return throw ref new Platform::FailureException();
    }
}
else if ((arraySize > D3D11_REQ_TEXTURE2D_ARRAY_AXIS_DIMENSION)
||
        (width > D3D11_REQ_TEXTURE2D_U_OR_V_DIMENSION) ||
        (height > D3D11_REQ_TEXTURE2D_U_OR_V_DIMENSION))
    {
        return throw ref new Platform::FailureException();
    }
break;

case D3D11_RESOURCE_DIMENSION_TEXTURE3D:
    if ((arraySize > 1) ||
        (width > D3D11_REQ_TEXTURE3D_U_V_OR_W_DIMENSION) ||
        (height > D3D11_REQ_TEXTURE3D_U_V_OR_W_DIMENSION) ||
        (depth > D3D11_REQ_TEXTURE3D_U_V_OR_W_DIMENSION))
    {
        return throw ref new Platform::FailureException();
    }
break;
}

// Create the texture.
std::unique_ptr<D3D11_SUBRESOURCE_DATA[]> initData(new
D3D11_SUBRESOURCE_DATA[mipCount * arraySize]);

size_t skipMip = 0;
size_t twidth = 0;
size_t theight = 0;
size_t tdepth = 0;
FillInitData(width, height, depth, mipCount, arraySize, format, maxsize,
bitSize, bitData, twidth, theight, tdepth, skipMip, initData.get());

hr = CreateD3DResources(d3dDevice, resDim, twidth, theight, tdepth,
mipCount - skipMip, arraySize, format, isCubeMap, initData.get(), texture,
textureView);

if (FAILED(hr) && !maxsize && (mipCount > 1))
{
    // Retry with a maxsize determined by feature level.
    switch (d3dDevice->GetFeatureLevel())
    {
    case D3D_FEATURE_LEVEL_9_1:
    case D3D_FEATURE_LEVEL_9_2:
        if (isCubeMap)
        {
            maxsize = D3D_FL9_1_REQ_TEXTURECUBE_DIMENSION;
        }
        else
        {
            maxsize = (resDim == D3D11_RESOURCE_DIMENSION_TEXTURE3D)

```

```

        ? D3D_FL9_1_REQ_TEXTURE3D_U_V_OR_W_DIMENSION
        : D3D_FL9_1_REQ_TEXTURE2D_U_OR_V_DIMENSION;
    }
    break;

    case D3D_FEATURE_LEVEL_9_3:
        maxsize = (resDim == D3D11_RESOURCE_DIMENSION_TEXTURE3D)
            ? D3D_FL9_1_REQ_TEXTURE3D_U_V_OR_W_DIMENSION
            : D3D_FL9_3_REQ_TEXTURE2D_U_OR_V_DIMENSION;
        break;

    default: // D3D_FEATURE_LEVEL_10_0 & D3D_FEATURE_LEVEL_10_1
        maxsize = (resDim == D3D11_RESOURCE_DIMENSION_TEXTURE3D)
            ? D3D10_REQ_TEXTURE3D_U_V_OR_W_DIMENSION
            : D3D10_REQ_TEXTURE2D_U_OR_V_DIMENSION;
        break;
}

FillInitData(width, height, depth, mipCount, arraySize, format,
maxsize, bitSize, bitData, twidth, theight, tdepth, skipMip,
initData.get());

hr = CreateD3DResources(d3dDevice, resDim, twidth, theight, tdepth,
mipCount - skipMip, arraySize, format, isCubeMap, initData.get(), texture,
textureView);
}

DX::ThrowIfFailed(hr);
}

//-----
void CreateDDSTextureFromMemory(
    _In_ ID3D11Device* d3dDevice,
    _In_reads_bytes_(ddsDataSize) const byte* ddsData,
    _In_ size_t ddsDataSize,
    _Out_opt_ ID3D11Resource** texture,
    _Out_opt_ ID3D11ShaderResourceView** textureView,
    _In_ size_t maxsize
)
{
    if (!d3dDevice || !ddsData || (!texture && !textureView))
    {
        throw ref new Platform::InvalidArgumentException();
    }

    // Validate DDS file in memory.
    if (ddsDataSize < (sizeof(uint32) + sizeof(DDS_HEADER)))
    {
        throw ref new Platform::FailureException();
    }

    uint32 dwMagicNumber = *(const uint32*)(ddsData);
    if (dwMagicNumber != DDS_MAGIC)
    {

```



```

        throw ref new Platform::FailureException();
    }

    const DDS_HEADER* header = reinterpret_cast<const DDS_HEADER*>(ddsData +
sizeof(uint32));

    // Verify the header to validate the DDS file.
    if (header->size != sizeof(DDS_HEADER) ||
        header->ddspf.size != sizeof(DDS_PIXELFORMAT))
    {
        throw ref new Platform::FailureException();
    }

    // Check for the DX10 extension.
    bool bDXT10Header = false;
    if ((header->ddspf.flags & DDS_FOURCC) &&
        (MAKEFOURCC('D', 'X', '1', '0') == header->ddspf.fourCC))
    {
        // Must be long enough for both headers and magic value
        if (ddsDataSize < (sizeof(DDS_HEADER) + sizeof(uint32) +
sizeof(DDS_HEADER_DXT10)))
        {
            throw ref new Platform::FailureException();
        }

        bDXT10Header = true;
    }

    ptrdiff_t offset = sizeof(uint32) + sizeof(DDS_HEADER) + (bDXT10Header ?
sizeof(DDS_HEADER_DXT10) : 0);

    CreateTextureFromDDS(d3dDevice, header, ddsData + offset, ddsDataSize -
offset, texture, textureView, maxsize);
}

```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Add features to DirectX games

Article • 10/20/2022

This section provides information about adding various features to your DirectX games.

[DirectX and XAML interop](#) topic explains how to use Extensible Application Markup Language (XAML) and DirectX together to build user interface frameworks.

[Game input](#) topic provides information about adding various types of controls to your game.

[Supporting screen orientation](#) topic discusses the best practices for handling screen rotation in your UWP DirectX game.

Topic	Description
DirectX and XAML interop	You can use Extensible Application Markup Language (XAML) together with Microsoft DirectX together in your Universal Windows Platform (UWP) game.
Game input	View links to topics about adding input to your DirectX games from move-look controls, relative mouse movement, and basic touch controls.
Supporting screen orientation	Best practices for handling screen rotation in your UWP DirectX app.

DirectX and XAML interop

Article • 04/25/2023

ⓘ Note

This topic applies to Universal Windows Platform (UWP) games and apps, and to types in the **Windows.UI.Xaml.Xxx** namespaces (not **Microsoft.UI.Xaml.Xxx**).

You can use Extensible Application Markup Language (XAML) together with Microsoft DirectX together in your Universal Windows Platform (UWP) game or app. The combination of XAML and DirectX lets you build flexible user interface frameworks that interoperate with your DirectX-rendered content; which is particularly useful for graphics-intensive apps. This topic explains the structure of a UWP app that uses DirectX, and identifies the important types to use when building your UWP app to work with DirectX.

If your app mainly focuses on 2D rendering, then you might want to use the [Win2D](#) Windows Runtime library. That library is maintained by Microsoft, and is built on top of the core [Direct2D](#) technology. Win2D greatly simplifies the usage pattern to implement 2D graphics, and includes helpful abstractions for some of the techniques described in this document. See the project page for more details. This document covers guidance for app developers who choose *not* to use Win2D.

ⓘ Note

DirectX APIs are not defined as Windows Runtime types, but you can use C++/WinRT to develop XAML UWP apps that interoperate with DirectX. If you factor the code that calls DirectX into its own C++/WinRT **Windows Runtime component** (WRC), then you can use that WRC in a UWP app (even a C# one) that then combines XAML and DirectX.

XAML and DirectX

DirectX provides two powerful libraries for 2D and 3D graphics, respectively—Direct2D and Direct3D. Although XAML provides support for basic 2D primitives and effects, many modeling and gaming apps need more complex graphics support. For these, you can use Direct2D and Direct3D to render the more complex graphics, and use XAML for more traditional user interface (UI) elements.

If you're implementing custom XAML and DirectX interop, then you need to know these two concepts.

- Shared surfaces are sized regions of the display, defined by XAML, that you can use DirectX to draw into indirectly by using [Windows::UI::Xaml::Media::ImageSource](#) types. For shared surfaces, you don't control the precise timing of when new content appears on-screen. Rather, updates to the shared surface are sync'd to the XAML framework's updates.
- A [swap chain](#) represents a collection of buffers that are used to display graphics with minimal latency. Typically, a swap chain is updated at 60 frames per second separately from the UI thread. However, a swap chain uses more memory and CPU resources in order to support rapid updates, and is relatively difficult to use, since you have to manage multiple threads.

Consider what you're using DirectX for. Will you use it to composite or animate a single control that fits within the dimensions of the display window? Will it contain output that needs to be rendered and controlled in real-time, as in a game? If so, then you'll probably need to implement a swap chain. Otherwise, you should be fine using a shared surface.

Once you've determined how you intend to use DirectX, you use one of the following Windows Runtime types to incorporate DirectX rendering into your UWP app.

- If you want to compose a static image, or draw a complex image at event-driven intervals, then draw to a shared surface with [Windows::UI::Xaml::Media::Imaging::SurfacelImageSource](#). That type handles a sized DirectX drawing surface. Typically, you use this type when composing an image or texture as a bitmap for display in a document or UI element. It doesn't work well for real-time interactivity, such as a high-performance game. That's because updates to a **SurfacelImageSource** object are synced to XAML user interface updates, and that can introduce latency into the visual feedback you provide to the user, such as a fluctuating frame rate, or a perceived poor response to real-time input. Updates are still quick enough for dynamic controls or data simulations, though.
- If the image is larger than the provided screen real estate, and can be panned or zoomed by the user, then use [Windows::UI::Xaml::Media::Imaging::VirtualSurfacelImageSource](#). That type handles a sized DirectX drawing surface that is larger than the screen. Like [SurfacelImageSource](#), you use this when composing a complex image or control dynamically. And, also like **SurfacelImageSource**, it doesn't work well for high-performance games. Some examples of XAML elements that could use a

VirtualSurfaceImageSource are map controls, or a large, image-dense document viewer.

- If you're using DirectX to present graphics updated in real-time, or in a situation where the updates must come on regular low-latency intervals, then use the **SwapChainPanel** class, so that you can refresh the graphics without syncing to the XAML framework refresh timer. With **SwapChainPanel** you can access the graphics device's swap chain (**IDXGISwapChain1**) directly, and layer XAML on top of the render target. **SwapChainPanel** works great for games and full-screen DirectX apps that require a XAML-based user interface. You must know DirectX well to use this approach, including the [Microsoft DirectX Graphics Infrastructure](#) (DXGI), Direct2D, and Direct3D technologies. For more info, see [Programming Guide for Direct3D 11](#).

SurfaceImageSource

SurfaceImageSource provides you with a DirectX shared surface to draw into; it then composes the bits into app content.

💡 Tip

The [Line spacing \(DirectWrite\)](#) and [Downloadable fonts \(DirectWrite\)](#) sample applications demonstrate **SurfaceImageSource**.

At a very high level, here's the process for creating and updating a **SurfaceImageSource**.

- Create a Direct 3D device, a Direct 2D device, and a Direct 2D device context.
- Create a **SurfaceImageSource**, and set the Direct 2D (or Direct 3D) device on that.
- Begin drawing on the **SurfaceImageSource** in order to obtain a DXGI surface.
- Draw to the DXGI surface with Direct2D (or Direct3D).
- End drawing on the **SurfaceImageSource** when you're done.
- Set the **SurfaceImageSource** on a XAML **Image** or **ImageBrush** in order to display it in your XAML UI.

And here's a deeper dive into those steps, with source code examples.

1. You can follow along with the code shown and described below by creating a new project in Microsoft Visual Studio. Create a **Blank App (C++/WinRT)** project. Target the latest generally-available (that is, not preview) version of the Windows SDK.
2. Open `pch.h`, and *add* the following includes below the ones already there.

C++/WinRT

```
// pch.h
...
#include <d3d11_4.h>
#include <d2d1_1.h>
#include <windows.ui.xaml.media.dxinterop.h>
#include <winrt/Windows.UI.Xaml.Media.Imaging.h>
```

3. Add the `using` directive shown below to the top of `MainPage.cpp`, below the ones already there. Also in `MainPage.cpp`, replace the existing implementation of `MainPage::ClickHandler` with the listing shown below. The code creates a Direct 3D device, a Direct 2D device, and a Direct 2D device context. To do that, it calls `D3D11CreateDevice`, `D2D1CreateDevice`, and `ID2D1Device::CreateDeviceContext`.

C++/WinRT

```
// MainPage.cpp | paste this below the existing using directives
using namespace Windows::UI::Xaml::Media::Imaging;
```

C++/WinRT

```
// MainPage.cpp | paste this to replace the existing
MainPage::ClickHandler
void MainPage::ClickHandler(IInspectable const&, RoutedEventArgs
const&)
{
    myButton().Content(box_value(L"Clicked"));

    uint32_t creationFlags = D3D11_CREATE_DEVICE_BGRA_SUPPORT;

    D3D_FEATURE_LEVEL featureLevels[] =
    {
        D3D_FEATURE_LEVEL_11_1,
        D3D_FEATURE_LEVEL_11_0,
        D3D_FEATURE_LEVEL_10_1,
        D3D_FEATURE_LEVEL_10_0,
        D3D_FEATURE_LEVEL_9_3,
        D3D_FEATURE_LEVEL_9_2,
        D3D_FEATURE_LEVEL_9_1
    };

    // Create the Direct3D device.
    winrt::com_ptr<::ID3D11Device> d3dDevice;
    D3D_FEATURE_LEVEL supportedFeatureLevel;
    winrt::check_hresult(::D3D11CreateDevice(
        nullptr,
        D3D_DRIVER_TYPE_HARDWARE,
        0,
        creationFlags,
        featureLevels,
```

```

        ARRAYSIZE(featureLevels),
        D3D11_SDK_VERSION,
        d3dDevice.put(),
        &supportedFeatureLevel,
        nullptr)
    );

    // Get the DXGI device.
    winrt::com_ptr<::IDXGIDevice> dxgiDevice{
        d3dDevice.as<::IDXGIDevice>() };

    // Create the Direct2D device and a corresponding context.
    winrt::com_ptr<::ID2D1Device> d2dDevice;
    ::D2D1CreateDevice(dxgiDevice.get(), nullptr, d2dDevice.put());

    winrt::com_ptr<::ID2D1DeviceContext> d2dDeviceContext;
    winrt::check_hresult(
        d2dDevice->CreateDeviceContext(
            D2D1_DEVICE_CONTEXT_OPTIONS_NONE,
            d2dDeviceContext.put()
        )
    );
}

```

4. Next, add code to create a [SurfaceImageSource](#), and set the Direct 2D (or Direct 3D) device on that by calling [ISurfaceImageSourceNativeWithD2D::SetDevice](#).

⚠ Note

If you'll be drawing to your **SurfaceImageSource** from a background thread, then you'll also need to ensure that the DXGI device has multi-threaded access enabled (as shown in the code below). For performance reasons you should do that *only* if you'll be drawing from a background thread.

Define the size of the shared surface by passing the height and width to the [SurfaceImageSource](#) constructor. You can also indicate whether the surface needs alpha (opacity) support.

To set the device, and run the draw operations, we'll need a pointer to [ISurfaceImageSourceNativeWithD2D](#). To obtain one, query the **SurfaceImageSource** object for its underlying **ISurfaceImageSourceNativeWithD2D** interface.

C++/WinRT

```

// MainPage.cpp | paste this at the end of MainPage::ClickHandler
SurfaceImageSource surfaceImageSource(500, 500);

```

```
winrt::com_ptr<::ISurfaceImageSourceNativeWithD2D> sisNativeWithD2D{
    surfaceImageSource.as<::ISurfaceImageSourceNativeWithD2D>() };

// Associate the Direct2D device with the SurfaceImageSource.
sisNativeWithD2D->SetDevice(d2dDevice.get());

// To enable multi-threaded access (optional)
winrt::com_ptr<::ID3D11Multithread> d3dMultiThread{
    d3dDevice.as<::ID3D11Multithread>() };
d3dMultiThread->SetMultithreadProtected(true);
```

5. Call **ISurfaceImageSourceNativeWithD2D::BeginDraw** to retrieve a DXGI surface (an **IDXGISurface** interface). You can call **ISurfaceImageSourceNativeWithD2D::BeginDraw** (and the later drawing commands) from a background thread if you've enabled multi-threaded access. In this step you also create a bitmap from the DXGI surface, and set that into the Direct 2D device context.

In the *offset* parameter, **ISurfaceImageSourceNativeWithD2D::BeginDraw** returns the point offset (an x,y value) of the updated target rectangle. You can use that offset to determine where to draw your updated content with the **ID2D1DeviceContext**.

C++/WinRT

```
// MainPage.cpp | paste this at the end of MainPage::ClickHandler
winrt::com_ptr<::IDXGISurface> dxgiSurface;

RECT updateRect{ 0, 0, 500, 500 };
POINT offset{ 0, 0 };
HRESULT beginDrawHR = sisNativeWithD2D->BeginDraw(
    updateRect,
    __uuidof(::IDXGISurface),
    dxgiSurface.put_void(),
    &offset);

// Create render target.
winrt::com_ptr<::ID2D1Bitmap1> bitmap;
winrt::check_hresult(
    d2dDeviceContext->CreateBitmapFromDxgiSurface(
        dxgiSurface.get(),
        nullptr,
        bitmap.put()
    )
);

// Set context's render target.
d2dDeviceContext->SetTarget(bitmap.get());
```


6. Use the Direct 2D device context to draw the contents of the **SurfaceImageSource**. Only the area specified for update in the previous step in the *updateRect* parameter is drawn.

C++/WinRT

```
// MainPage.cpp | paste this at the end of MainPage::ClickHandler
if (beginDrawHR == DXGI_ERROR_DEVICE_REMOVED ||
    beginDrawHR == DXGI_ERROR_DEVICE_RESET ||
    beginDrawHR == D2DERR_RECREATE_TARGET)
{
    // The Direct3D and Direct2D devices were lost and need to be re-
    // created.
    // Recovery steps are:
    // 1) Re-create the Direct3D and Direct2D devices
    // 2) Call ISurfaceImageSourceNativeWithD2D::SetDevice with the new
    Direct2D
    //     device
    // 3) Redraw the contents of the SurfaceImageSource
}
else if (beginDrawHR == E_SURFACE_CONTENTS_LOST)
{
    // The devices were not lost but the entire contents of the surface
    // were. Recovery steps are:
    // 1) Call ISurfaceImageSourceNativeWithD2D::SetDevice with the
    Direct2D
    //     device again
    // 2) Redraw the entire contents of the SurfaceImageSource
}
else
{
    // Draw using Direct2D context.
    d2dDeviceContext->BeginDraw();

    d2dDeviceContext->Clear(D2D1::ColorF(D2D1::ColorF::Orange));

    winrt::com_ptr<ID2D1SolidColorBrush> brush;
    winrt::check_hresult(d2dDeviceContext->CreateSolidColorBrush(
        D2D1::ColorF(D2D1::ColorF::Chocolate),
        D2D1::BrushProperties(0.8f),
        brush.put()));

    D2D1_SIZE_F const size{ 500, 500 };
    D2D1_RECT_F const rect{ 100.0f, 100.0f, size.width - 100.0f,
        size.height - 100.0f };
    d2dDeviceContext->DrawRectangle(rect, brush.get(), 100.0f);

    d2dDeviceContext->EndDraw();
}
```

7. Call **ISurfaceImageSourceNativeWithD2D::EndDraw** to complete the bitmap (you must call **ISurfaceImageSourceNativeWithD2D::EndDraw** only from the UI thread).

Then set the **SurfaceImageSource** on a XAML **Image** (or **ImageBrush**) in order to display it in your XAML UI.

C++/WinRT

```
// MainPage.cpp | paste this at the end of MainPage::ClickHandler
sisNativeWithD2D->EndDraw();

// The SurfaceImageSource object's underlying
// ISurfaceImageSourceNativeWithD2D object will contain the completed
// bitmap.

theImage().Source(surfaceImageSource);
```

⚠ Note

Calling **SurfaceImageSource::SetSource** (inherited from **IBitmapSource::SetSource**) currently throws an exception. Don't call it from your **SurfaceImageSource** object.

⚠ Note

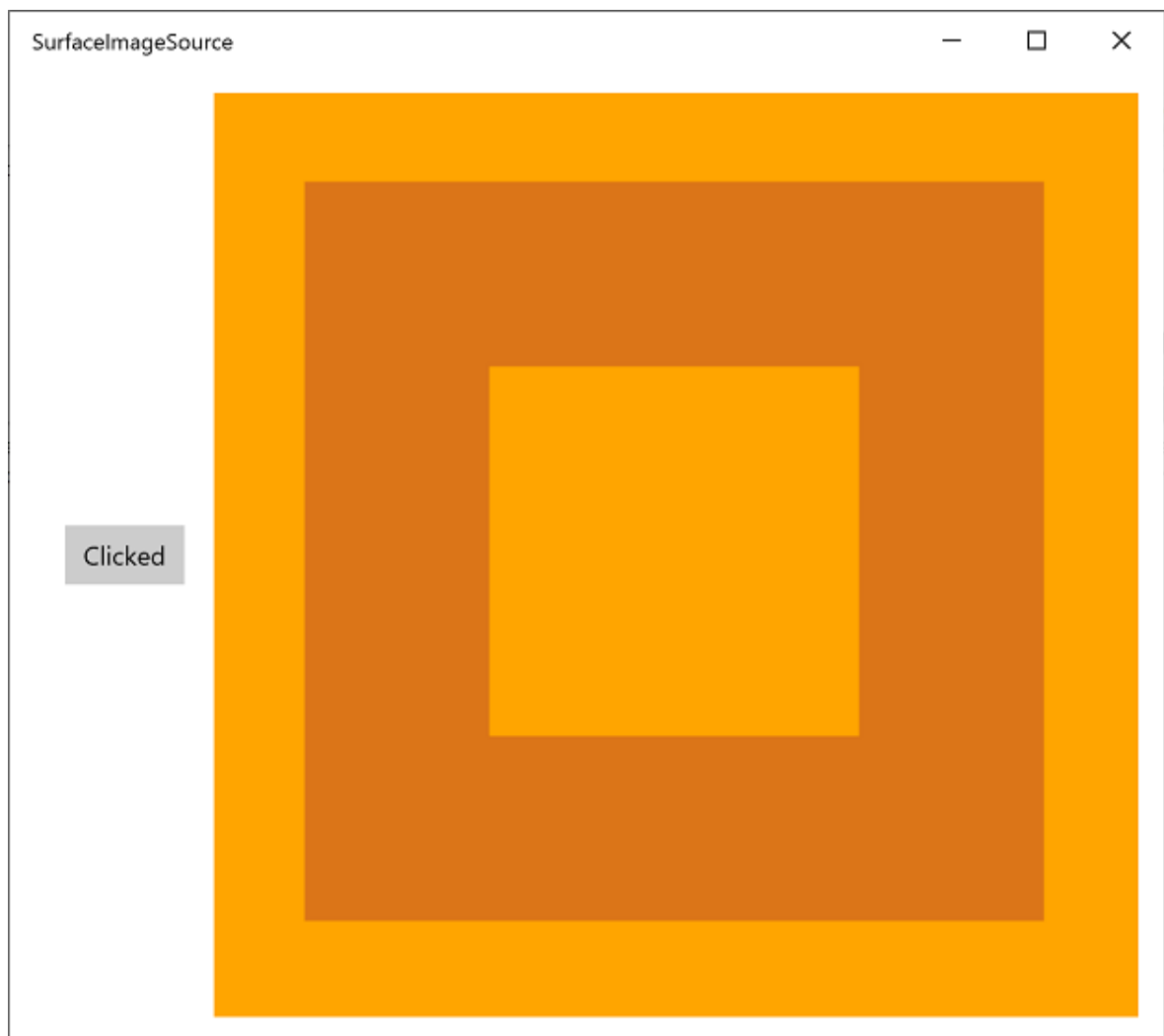
Avoid drawing to **SurfaceImageSource** while your **Window** is hidden or inactive, otherwise **ISurfaceImageSourceNativeWithD2D** APIs will fail. Handle events around window visibility and application suspension to accomplish that.

8. Finally, add the following **Image** element inside the existing XAML markup in `MainPage.xaml`.

XAML

```
<!-- MainPage.xaml -->
...
<Image x:Name="theImage" Width="500" Height="500" />
...
```

9. You can now build and run the app. Click the button to see the contents of the **SurfaceImageSource** displayed in the **Image**.



VirtualSurfacelmageSource

[VirtualSurfacelmageSource](#) extends [SurfacelmageSource](#), and it's for scenarios where the content is potentially too large to fit on the screen all at once (and/or too large to fit in video memory as a single texture), and so the content must be virtualized in order to render optimally. For example, mapping apps, or large document canvases.

💡 Tip

The **Complex inking sample** application demonstrates [VirtualSurfacelmageSource](#).

[VirtualSurfacelmageSource](#) differs from [SurfacelmageSource](#) in that it uses a callback—[IVirtualSurfacelmageSourceCallbacksNative::UpdatesNeeded](#)—which you implement to update regions of the surface as they become visible on the screen. You don't need to clear regions that are hidden, because the XAML framework takes care of that for you.

It's a good idea to become familiar with [SurfacelmageSource](#) (see the [SurfacelmageSource](#) section above) before tackling [VirtualSurfacelmageSource](#). But at

a very high level, here's the process for creating and updating a **VirtualSurfaceImageSource**.

- Implement the **IVirtualSurfaceImageSourceCallbackNative** interface.
- Create a Direct 3D device, a Direct 2D device, and a Direct 2D device context.
- Create a **VirtualSurfaceImageSource**, and set the Direct 2D (or Direct 3D) device on that.
- Call **RegisterForUpdatesNeeded** on the **VirtualSurfaceImageSource**.
- In your **UpdatesNeeded** callback, call **GetUpdateRectCount** and **GetUpdateRects**.
- Render the update rectangles (using **BeginDraw/EndDraw** just like for **SurfaceImageSource**).
- Set the **SurfaceImageSource** on a XAML **Image** or **ImageBrush** in order to display it in your XAML UI.

And here's a deeper dive into those steps, with source code examples.

1. You can follow along with the code shown and described below by creating a new project in Microsoft Visual Studio. Create a **Blank App (C++/WinRT)** project, and name it *VSISDemo* (it's important to give the project this name if you'll be copy-pasting in the code listings given below). Target the latest generally-available (that is, not preview) version of the Windows SDK.
2. Open `pch.h`, and *add* the following includes below the ones already there.

C++/WinRT

```
// pch.h
...
#include <d3d11_4.h>
#include <d2d1_1.h>
#include <windows.ui.xaml.media.dxinterop.h>
#include <winrt/Windows.UI.Xaml.Media.Imaging.h>
```

3. In this step, you'll provide an implementation of the **IVirtualSurfaceUpdatesCallbackNative** interface. Add a new **Header File (.h)** item to the project, and name it *CallbackImplementation.h*. Replace the contents of that file with the listing below. The code is explained after the listing.

C++/WinRT

```
#include "pch.h"

namespace winrt::VSISDemo::implementation
{
    struct CallbackImplementation :
        winrt::implements<CallbackImplementation,
```

```

::IVirtualSurfaceUpdatesCallbackNative>
{
    CallbackImplementation(
        winrt::com_ptr<::ISurfaceImageSourceNativeWithD2D>
sisNativeWithD2D,
        winrt::com_ptr<::IVirtualSurfaceImageSourceNative> const&
vsisNative,
        winrt::com_ptr<::ID2D1DeviceContext> const&
d2dDeviceContext) :
        m_sisNativeWithD2D(sisNativeWithD2D),
        m_vsisNative(vsisNative),
        m_d2dDeviceContext(d2dDeviceContext)
    {}

    IFACEMETHOD(UpdatesNeeded)()
    {
        HRESULT hr = S_OK;

        ULONG drawingBoundsCount = 0;
        m_vsisNative->GetUpdateRectCount(&drawingBoundsCount);

        std::unique_ptr<RECT[]> drawingBounds(
            new RECT[drawingBoundsCount]);

        m_vsisNative->GetUpdateRects(
            drawingBounds.get(),
            drawingBoundsCount);

        for (ULONG i = 0; i < drawingBoundsCount; ++i)
        {
            winrt::com_ptr<::IDXGISurface> dxgiSurface;

            POINT offset{ 0, 0 };
            HRESULT beginDrawHR = m_sisNativeWithD2D->BeginDraw(
                drawingBounds[i],
                __uuidof(::IDXGISurface),
                dxgiSurface.put_void(),
                &offset);

            // Create render target.
            winrt::com_ptr<::ID2D1Bitmap1> bitmap;
            winrt::check_hresult(
                m_d2dDeviceContext->CreateBitmapFromDxgiSurface(
                    dxgiSurface.get(),
                    nullptr,
                    bitmap.put()
                )
            );

            // Set context's render target.
            m_d2dDeviceContext->SetTarget(bitmap.get());

            if (beginDrawHR == DXGI_ERROR_DEVICE_REMOVED ||
                beginDrawHR == DXGI_ERROR_DEVICE_RESET ||
                beginDrawHR == D2DERR_RECREATE_TARGET)

```

```

        {
            // The Direct3D and Direct2D devices were lost and
            need to be re-created.
            // Recovery steps are:
            // 1) Re-create the Direct3D and Direct2D devices
            // 2) Call
            ISurfaceImageSourceNativeWithD2D::SetDevice with the new Direct2D
            //     device
            // 3) Redraw the contents of the SurfaceImageSource
        }
        else if (beginDrawHR == E_SURFACE_CONTENTS_LOST)
        {
            // The devices were not lost but the entire
            contents of the surface
            // were. Recovery steps are:
            // 1) Call
            ISurfaceImageSourceNativeWithD2D::SetDevice with the Direct2D
            //     device again
            // 2) Redraw the entire contents of the
            SurfaceImageSource
        }
        else
        {
            // Draw using Direct2D context.
            m_d2dDeviceContext->BeginDraw();

            m_d2dDeviceContext->
            >Clear(D2D1::ColorF(D2D1::ColorF::Orange));

            winrt::com_ptr<::ID2D1SolidColorBrush> brush;
            winrt::check_hresult(m_d2dDeviceContext->
            >CreateSolidColorBrush(
                D2D1::ColorF(D2D1::ColorF::Chocolate),
                D2D1::BrushProperties(0.8f),
                brush.put()));

            D2D1_SIZE_F const size{ drawingBounds[i].right -
            drawingBounds[i].left, drawingBounds[i].bottom - drawingBounds[i].top
            };
            D2D1_RECT_F const rect{ 100.0f, 100.0f, size.width
            - 100.0f, size.height - 100.0f };
            m_d2dDeviceContext->DrawRectangle(rect,
            brush.get(), 100.0f);

            m_d2dDeviceContext->EndDraw();
        }

        m_sisNativeWithD2D->EndDraw();
    }

    return hr;
}

private:
    winrt::com_ptr<::ISurfaceImageSourceNativeWithD2D>

```

```

m_sisNativeWithD2D{ nullptr };
    winrt::com_ptr<::IVirtualSurfaceImageSourceNative>
m_vsisNative{ nullptr };
    winrt::com_ptr<::ID2D1DeviceContext> m_d2dDeviceContext{
    nullptr };
};
}

```

Whenever a region of the **VirtualSurfaceImageSource** needs to be updated, the framework calls your implementation of

IVirtualSurfaceUpdatesCallbackNative::UpdatesNeeded (shown above).

That can happen either when the framework determines that the region needs to be drawn (when the user pans or zooms the view of the surface, for example), or after your app has called **IVirtualSurfaceImageSourceNative::Invalidate** on that region.

In your implementation of **IVirtualSurfaceImageSourceNative::UpdatesNeeded**, use the **IVirtualSurfaceImageSourceNative::GetUpdateRectCount** and **IVirtualSurfaceImageSourceNative::GetUpdateRects** methods to determine which region(s) of the surface must be drawn.

For each region that must be updated, draw the specific content to that region, but constrain your drawing to the bounded regions for better performance. The specifics of calling the **ISurfaceImageSourceNativeWithD2D** methods are the same as for **SurfaceImageSource** (see the **SurfaceImageSource** section above).

⚠ Note

Avoid drawing to **VirtualSurfaceImageSource** while your **Window** is hidden or inactive, otherwise **ISurfaceImageSourceNativeWithD2D** APIs will fail. Handle events around window visibility and application suspension to accomplish that.

4. In the **MainPage** class, we'll add a member of type **CallbackImplementation**. We'll also create a Direct 3D device, a Direct 2D device, and a Direct 2D device context. To do that, we'll call **D3D11CreateDevice**, **D2D1CreateDevice**, and **ID2D1Device::CreateDeviceContext**.

Replace the contents of **MainPage.idl**, **MainPage.h**, and **MainPage.cpp** with the contents of the listings below.

```
// MainPage.idl
namespace VSISDemo
{
    [default_interface]
    runtimeclass MainPage : Windows.UI.Xaml.Controls.Page
    {
        MainPage();
    }
}
```

C++/WinRT

```
// MainPage.h
#pragma once

#include "MainPage.g.h"
#include "CallbackImplementation.h"

namespace winrt::VSISDemo::implementation
{
    struct MainPage : MainPageT<MainPage>
    {
        MainPage();
        void ClickHandler(Windows::Foundation::IInspectable const&
sender, Windows::UI::Xaml::RoutedEventArgs const& args);

        private:
            winrt::com_ptr<::IVirtualSurfaceUpdatesCallbackNative> m_cbi{
nullptr };
    };
}

namespace winrt::VSISDemo::factory_implementation
{
    struct MainPage : MainPageT<MainPage, implementation::MainPage>
    {
    };
}
```

C++/WinRT

```
// MainPage.cpp
#include "pch.h"
#include "MainPage.h"
#include "MainPage.g.cpp"

using namespace winrt;
using namespace Windows::UI::Xaml;
using namespace Windows::UI::Xaml::Media::Imaging;

namespace winrt::VSISDemo::implementation
```



```

{
    MainPage::MainPage()
    {
        InitializeComponent();
    }

    void MainPage::ClickHandler(IInspectable const&, RoutedEventArgs
const&)
    {
        myButton().Content(box_value(L"Clicked"));

        uint32_t creationFlags = D3D11_CREATE_DEVICE_BGRA_SUPPORT;

        D3D_FEATURE_LEVEL featureLevels[] =
        {
            D3D_FEATURE_LEVEL_11_1,
            D3D_FEATURE_LEVEL_11_0,
            D3D_FEATURE_LEVEL_10_1,
            D3D_FEATURE_LEVEL_10_0,
            D3D_FEATURE_LEVEL_9_3,
            D3D_FEATURE_LEVEL_9_2,
            D3D_FEATURE_LEVEL_9_1
        };

        // Create the Direct3D device.
        winrt::com_ptr<::ID3D11Device> d3dDevice;
        D3D_FEATURE_LEVEL supportedFeatureLevel;
        winrt::check_hresult(::D3D11CreateDevice(
            nullptr,
            D3D_DRIVER_TYPE_HARDWARE,
            0,
            creationFlags,
            featureLevels,
            ARRAYSIZE(featureLevels),
            D3D11_SDK_VERSION,
            d3dDevice.put(),
            &supportedFeatureLevel,
            nullptr)
        );

        // Get the Direct3D device.
        winrt::com_ptr<::IDXGIDevice> dxgiDevice{
            d3dDevice.as<::IDXGIDevice>() };

        // Create the Direct2D device and a corresponding context.
        winrt::com_ptr<::ID2D1Device> d2dDevice;
        ::D2D1CreateDevice(dxgiDevice.get(), nullptr, d2dDevice.put());

        winrt::com_ptr<::ID2D1DeviceContext> d2dDeviceContext;
        winrt::check_hresult(
            d2dDevice->CreateDeviceContext(
                D2D1_DEVICE_CONTEXT_OPTIONS_NONE,
                d2dDeviceContext.put()
            )
        );
    }
}

```

```
}  
}
```

5. Next, add code to create a [VirtualSurfaceImageSource](#) with the size that you want, and set the Direct 2D (or Direct 3D) device on that by calling [ISurfaceImageSourceNativeWithD2D::SetDevice](#).

⚠ Note

If you'll be drawing to your **VirtualSurfaceImageSource** from a background thread, then you'll also need to ensure that the DXGI device has multi-threaded access enabled (as shown in the code below). For performance reasons you should do that *only* if you'll be drawing from a background thread.

To set the device, and run the draw operations, we'll need a pointer to [ISurfaceImageSourceNativeWithD2D](#). To obtain one, query the **VirtualSurfaceImageSource** object for its underlying **ISurfaceImageSourceNativeWithD2D** interface.

Also query for [IVirtualSurfaceImageSourceNative](#), and call [IVirtualSurfaceImageSourceNative::RegisterForUpdatesNeeded](#), providing your implementation of [IVirtualSurfaceUpdatesCallbackNative](#).

Then set the **SurfaceImageSource** on a XAML [Image](#) (or [ImageBrush](#)) in order to display it in your XAML UI.

C++/WinRT

```
// MainPage.cpp | paste this at the end of MainPage::ClickHandler  
VirtualSurfaceImageSource virtualSurfaceImageSource(2000, 2000);  
  
winrt::com_ptr<::ISurfaceImageSourceNativeWithD2D> sisNativeWithD2D{  
    virtualSurfaceImageSource.as<::ISurfaceImageSourceNativeWithD2D>()  
};  
  
// Associate the Direct2D device with the SurfaceImageSource.  
sisNativeWithD2D->SetDevice(d2dDevice.get());  
  
// To enable multi-threaded access (optional)  
winrt::com_ptr<::ID3D11Multithread> d3dMultiThread{  
    d3dDevice.as<::ID3D11Multithread>() };  
d3dMultiThread->SetMultithreadProtected(true);  
  
winrt::com_ptr<::IVirtualSurfaceImageSourceNative> vsisNative{  
    virtualSurfaceImageSource.as<::IVirtualSurfaceImageSourceNative>()  
}
```

```
};

m_cbi = winrt::make<CallbackImplementation>(sisNativeWithD2D,
vsisNative, d2dDeviceContext);
vsisNative-
>RegisterForUpdatesNeeded(m_cbi.as<::IVirtualSurfaceUpdatesCallbackNati
ve>().get());

// The SurfaceImageSource object's underlying
// ISurfaceImageSourceNativeWithD2D object will contain the completed
bitmap.

theImage().Source(virtualSurfaceImageSource);
```

6. Finally, add the following **Image** element inside the existing XAML markup in `MainPage.xaml`.

```
XAML

<!-- MainPage.xaml -->
...
<Image x:Name="theImage" Width="500" Height="500" />
...
```

7. You can now build and run the app. Click the button to see the contents of the **VirtualSurfaceImageSource** displayed in the **Image**.

SwapChainPanel and gaming

SwapChainPanel is the Windows Runtime type designed to support high-performance graphics and gaming, where you manage the swap chain directly. In this case, you create your own DirectX swap chain and manage the presentation of your rendered content. Another feature of the **SwapChainPanel** is that you can overlay other XAML elements in front of it.

💡 Tip

The following sample applications demonstrate **SurfaceImageSource: Direct2D** advanced color image rendering, **Direct2D** photo adjustment, **Direct2D** SVG image rendering, **Low latency input**, **DirectX and XAML game**, and **XAML SwapChainPanel DirectX interop (Windows 8.1)** [↗](#).

To ensure good performance, there are certain limitations to the **SwapChainPanel** type.

- There should be no more than 4 **SwapChainPanel** instances per app.
- You should set the DirectX swap chain's height and width (in [DXGI_SWAP_CHAIN_DESC1](#)) to the current dimensions of the swap chain element. If you don't, then the display content will be scaled to fit (using [DXGI_SCALING_STRETCH](#)).
- You must set the DirectX swap chain's scaling mode (in [DXGI_SWAP_CHAIN_DESC1](#)) to [DXGI_SCALING_STRETCH](#).
- You must create the DirectX swap chain by calling [IDXGIFactory2::CreateSwapChainForComposition](#).

You update the [SwapChainPanel](#) based on the needs of your app, and not sync'd with the updates of the XAML framework. If you need to synchronize the updates of the **SwapChainPanel** to those of the XAML framework, then register for the [Windows::UI::Xaml::Media::CompositionTarget::Rendering](#) event. Otherwise, you must consider any cross-thread issues if you try to update the XAML elements from a different thread than the one updating the **SwapChainPanel**.

If you need to receive low-latency pointer input to your **SwapChainPanel**, then use [SwapChainPanel::CreateCoreIndependentInputSource](#). That method returns a [CoreIndependentInputSource](#) object that can be used to receive input events at minimal latency on a background thread. Note that once this method is called, normal XAML pointer input events will not be raised for the **SwapChainPanel**, since all input will be redirected to the background thread.

Here's the process for creating and updating a [SwapChainPanel](#) object.

1. You can follow along with the code shown and described below by creating a new project in Microsoft Visual Studio. Create a **Blank App (C++/WinRT)** project, and name it *SCPDemo* (it's important to give the project this name if you'll be copy-pasting in the code listings given below). Target the latest generally-available (that is, not preview) version of the Windows SDK.
2. Open `pch.h`, and *add* the following includes below the ones already there.

C++/WinRT

```
// pch.h
...
#include <d3d11_4.h>
#include <d2d1_1.h>
#include <windows.ui.xaml.media.dxinterop.h>
```

3. In the **MainPage** class, we'll first create a Direct 3D device, a Direct 2D device, and a Direct 2D device context. To do that, we'll call [D3D11CreateDevice](#),

[ID2D1CreateDevice](#), and [ID2D1Device::CreateDeviceContext](#).

Replace the contents of `MainPage.idl`, `MainPage.h`, and `MainPage.cpp` with the contents of the listings below.

C++/WinRT

```
// MainPage.idl
namespace SCPDemo
{
    [default_interface]
    runtimeclass MainPage : Windows.UI.Xaml.Controls.Page
    {
        MainPage();
    }
}
```

C++/WinRT

```
// MainPage.h
#pragma once

#include "MainPage.g.h"

namespace winrt::SCPDemo::implementation
{
    struct MainPage : MainPageT<MainPage>
    {
        MainPage();
        void ClickHandler(Windows::Foundation::IInspectable const&
sender, Windows::UI::Xaml::RoutedEventArgs const& args);
    };
}

namespace winrt::SCPDemo::factory_implementation
{
    struct MainPage : MainPageT<MainPage, implementation::MainPage>
    {
    };
}
```

C++/WinRT

```
// MainPage.cpp
#include "pch.h"
#include "MainPage.h"
#include "MainPage.g.cpp"

using namespace winrt;
using namespace Windows::UI::Xaml;
```

```

namespace winrt::SCPDemo::implementation
{
    MainPage::MainPage()
    {
        InitializeComponent();

        void MainPage::ClickHandler(IInspectable const&, RoutedEventArgs
const&)
        {
            myButton().Content(box_value(L"Clicked"));

            uint32_t creationFlags = D3D11_CREATE_DEVICE_BGRA_SUPPORT;

            D3D_FEATURE_LEVEL featureLevels[] =
            {
                D3D_FEATURE_LEVEL_11_1,
                D3D_FEATURE_LEVEL_11_0,
                D3D_FEATURE_LEVEL_10_1,
                D3D_FEATURE_LEVEL_10_0,
                D3D_FEATURE_LEVEL_9_3,
                D3D_FEATURE_LEVEL_9_2,
                D3D_FEATURE_LEVEL_9_1
            };

            // Create the Direct3D device.
            winrt::com_ptr<::ID3D11Device> d3dDevice;
            D3D_FEATURE_LEVEL supportedFeatureLevel;
            winrt::check_hresult(::D3D11CreateDevice(
                nullptr,
                D3D_DRIVER_TYPE_HARDWARE,
                0,
                creationFlags,
                featureLevels,
                ARRAYSIZE(featureLevels),
                D3D11_SDK_VERSION,
                d3dDevice.put(),
                &supportedFeatureLevel,
                nullptr)
            );

            // Get the Direct3D device.
            winrt::com_ptr<::IDXGIDevice> dxgiDevice{
                d3dDevice.as<::IDXGIDevice>() };

            // Create the Direct2D device and a corresponding context.
            winrt::com_ptr<::ID2D1Device> d2dDevice;
            ::D2D1CreateDevice(dxgiDevice.get(), nullptr, d2dDevice.put());

            winrt::com_ptr<::ID2D1DeviceContext> d2dDeviceContext;
            winrt::check_hresult(
                d2dDevice->CreateDeviceContext(
                    D2D1_DEVICE_CONTEXT_OPTIONS_NONE,
                    d2dDeviceContext.put()
                )
            );

```

```

    );
}
}

```

4. Wrap your XAML markup in a **SwapChainPanel** element with an `x:Name`. The wrapped XAML elements will render in front of the **SwapChainPanel**.

XAML

```

<!-- MainPage.xaml -->
<SwapChainPanel x:Name="swapChainPanel">
    <StackPanel Orientation="Horizontal" HorizontalAlignment="Center"
VerticalAlignment="Center">
        <Button x:Name="myButton" Click="ClickHandler">Click
Me</Button>
    </StackPanel>
</SwapChainPanel>

```

You can then access that **SwapChainPanel** object via the accessor function with the same name, as we'll see.

5. Next, call `IDXGIFactory2::CreateSwapChainForComposition` to create a swap chain.

C++/WinRT

```

// MainPage.cpp | paste this at the end of MainPage::ClickHandler
// Get the DXGI adapter.
winrt::com_ptr< ::IDXGIAdapter > dxgiAdapter;
dxgiDevice->GetAdapter(dxgiAdapter.put());

// Get the DXGI factory.
winrt::com_ptr< ::IDXGIFactory2 > dxgiFactory;
dxgiFactory.capture(dxgiAdapter, &IDXGIAdapter::GetParent);

DXGI_SWAP_CHAIN_DESC1 swapChainDesc { 0 };
swapChainDesc.Width = 500;
swapChainDesc.Height = 500;
swapChainDesc.Format = DXGI_FORMAT_B8G8R8A8_UNORM; // This is the most
common swapchain format.
swapChainDesc.Stereo = false;
swapChainDesc.SampleDesc.Count = 1; // Don't use multi-sampling.
swapChainDesc.SampleDesc.Quality = 0;
swapChainDesc.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
swapChainDesc.BufferCount = 2;
swapChainDesc.Scaling = DXGI_SCALING_STRETCH;
swapChainDesc.SwapEffect = DXGI_SWAP_EFFECT_FLIP_SEQUENTIAL; // We
recommend using this swap effect for all applications.
swapChainDesc.Flags = 0;

```

```
// Create a swap chain by calling
IDXGIFactory2::CreateSwapChainForComposition.
winrt::com_ptr< ::IDXGISwapChain1 > swapChain;
dxgiFactory->CreateSwapChainForComposition(
    d3dDevice.get(),
    &swapChainDesc,
    nullptr,
    swapChain.put());
```

6. Get an **ISwapChainPanelNative** from the **SwapChainPanel** that you named *swapChainPanel*. The call **ISwapChainPanelNative::SetSwapChain** to set the swap chain on the **SwapChainPanel**.

C++/WinRT

```
// MainPage.cpp | paste this at the end of MainPage::ClickHandler
// Get native interface for SwapChainPanel
auto panelNative{ swapChainPanel().as<ISwapChainPanelNative>() };

winrt::check_hresult(
    panelNative->SetSwapChain(swapChain.get())
);
```

7. Lastly, draw to the DirectX swap chain, and then present it to display the contents.

C++/WinRT

```
// Create a Direct2D target bitmap associated with the
// swap chain back buffer, and set it as the current target.
D2D1_BITMAP_PROPERTIES1 bitmapProperties =
    D2D1::BitmapProperties1(
        D2D1_BITMAP_OPTIONS_TARGET | D2D1_BITMAP_OPTIONS_CANNOT_DRAW,
        D2D1::PixelFormat(DXGI_FORMAT_B8G8R8A8_UNORM,
        D2D1_ALPHA_MODE_PREMULTIPLIED),
        96.f,
        96.f
    );

winrt::com_ptr<::IDXGISurface> dxgiBackBuffer;
swapChain->GetBuffer(0, __uuidof(dxgiBackBuffer),
dxgiBackBuffer.put_void());

winrt::com_ptr< ::ID2D1Bitmap1 > targetBitmap;
winrt::check_hresult(
    d2dDeviceContext->CreateBitmapFromDxgiSurface(
        dxgiBackBuffer.get(),
        &bitmapProperties,
        targetBitmap.put()
    )
);
```



```

d2dDeviceContext->SetTarget(targetBitmap.get());

// Draw using Direct2D context.
d2dDeviceContext->BeginDraw();

d2dDeviceContext->Clear(D2D1::ColorF(D2D1::ColorF::Orange));

winrt::com_ptr<ID2D1SolidColorBrush> brush;
winrt::check_hresult(d2dDeviceContext->CreateSolidColorBrush(
    D2D1::ColorF(D2D1::ColorF::Chocolate),
    D2D1::BrushProperties(0.8f),
    brush.put()));

D2D1_SIZE_F const size{ 500, 500 };
D2D1_RECT_F const rect{ 100.0f, 100.0f, size.width - 100.0f,
    size.height - 100.0f };
d2dDeviceContext->DrawRectangle(rect, brush.get(), 100.0f);

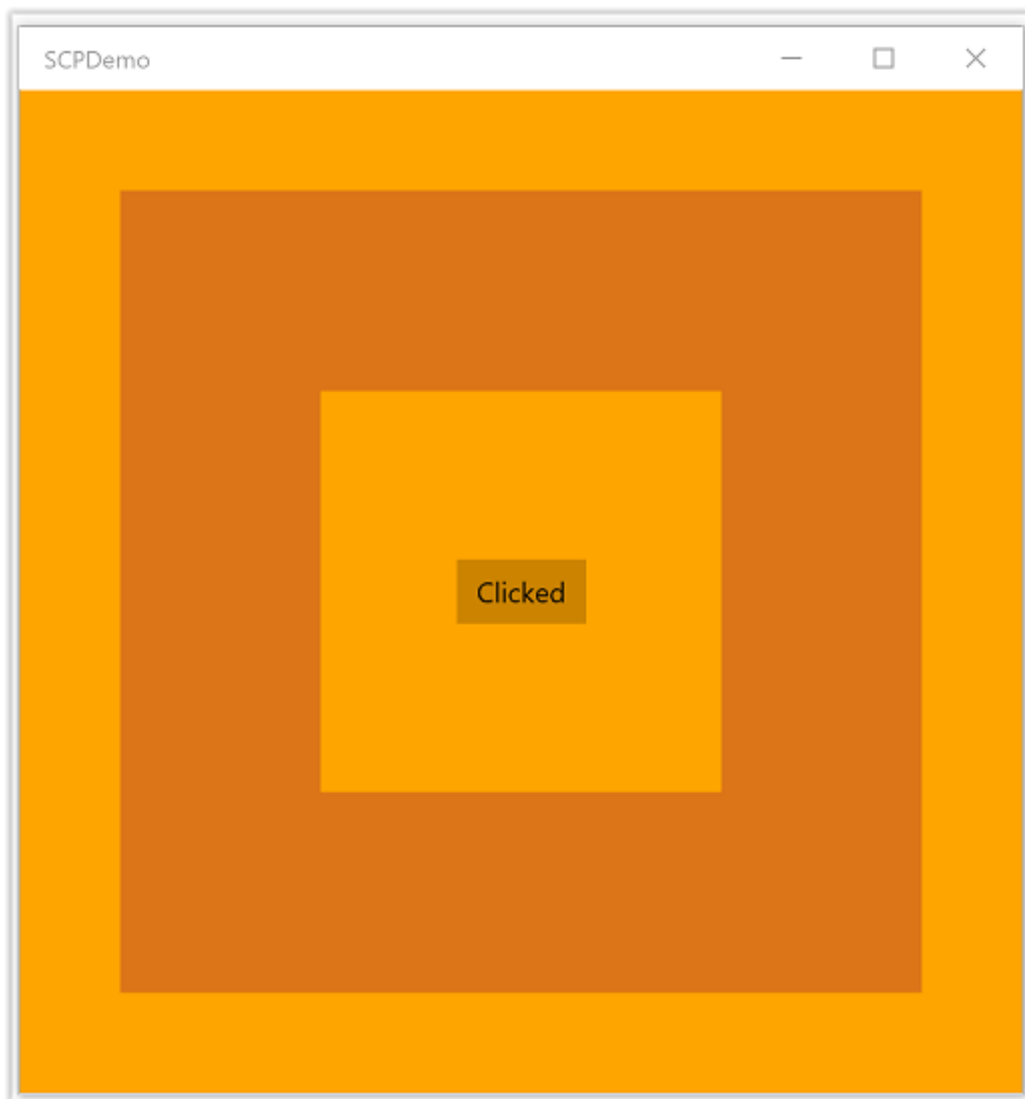
d2dDeviceContext->EndDraw();

swapChain->Present(1, 0);

```

The XAML elements are refreshed when the Windows Runtime layout/render logic signals an update.

8. You can now build and run the app. Click the button to see the contents of the **SwapChainPanel** displayed behind the other XAML elements.



ⓘ Note

In general, your DirectX app should create swap chains in landscape orientation, and equal to the display window size (which is usually the native screen resolution in most Microsoft Store games). That ensures that your app uses the optimal swap chain implementation when it doesn't have any visible XAML overlay. If the app is rotated to portrait mode, then your app should call **IDXGISwapChain1::SetRotation** on the existing swap chain, apply a transform to the content if needed, and then call **SetSwapChain** again on the same swap chain. Similarly, your app should call **SetSwapChain** again on the same swap chain whenever the swap chain is resized by calling **IDXGISwapChain::ResizeBuffers**.

Related topics

- [Win2D](#) 
- [SurfaceImageSource](#)
- [VirtualSurfaceImageSource](#)

- [SwapChainPanel](#)
- [ISwapChainPanelNative](#)
- [Programming Guide for Direct3D 11](#)

Game input for DirectX games

Article • 10/20/2022

This section provides information about adding various types of inputs for your DirectX games.

Move-look controls for games topic explains how to add traditional mouse and keyboard controls to your DirectX game.

Relative mouse movement topic discusses how you can implement mouse control by tracking the pixel delta between mouse movements.

Touch controls for games topic explains how to add touch-based controls to move a fixed-plane camera in a Direct3D environment.

Topic	Description
Move-look controls for games	Add move-look controls.
Relative mouse movement	Handle relative mouse movement.
Touch controls for games	Add basic touch controls to your game.

Move-look controls for games

Article • 10/20/2022

Learn how to add traditional mouse and keyboard move-look controls (also known as mouselook controls) to your DirectX game.

We also discuss move-look support for touch devices, with the move controller defined as the lower-left section of the screen that behaves like a directional input, and the look controller defined for the remainder of the screen, with the camera centering on the last place the player touched in that area.

If this is an unfamiliar control concept to you, think of it this way: the keyboard (or the touch-based directional input box) controls your legs in this 3D space, and behaves as if your legs were only capable of moving forward or backward, or strafing left and right. The mouse (or touch pointer) controls your head. You use your head to look in a direction -- left or right, up or down, or somewhere in that plane. If there is a target in your view, you would use the mouse to center your camera view on that target, and then press the forward key to move towards it, or back to move away from it. To circle the target, you would keep the camera view centered on the target, and move left or right at the same time. You can see how this is a very effective control method for navigating 3D environments!

These controls are commonly known as WASD controls in gaming, where the W, A, S, and D keys are used for x-z plane fixed camera movement, and the mouse is used to control camera rotation around the x and y axes.

Objectives

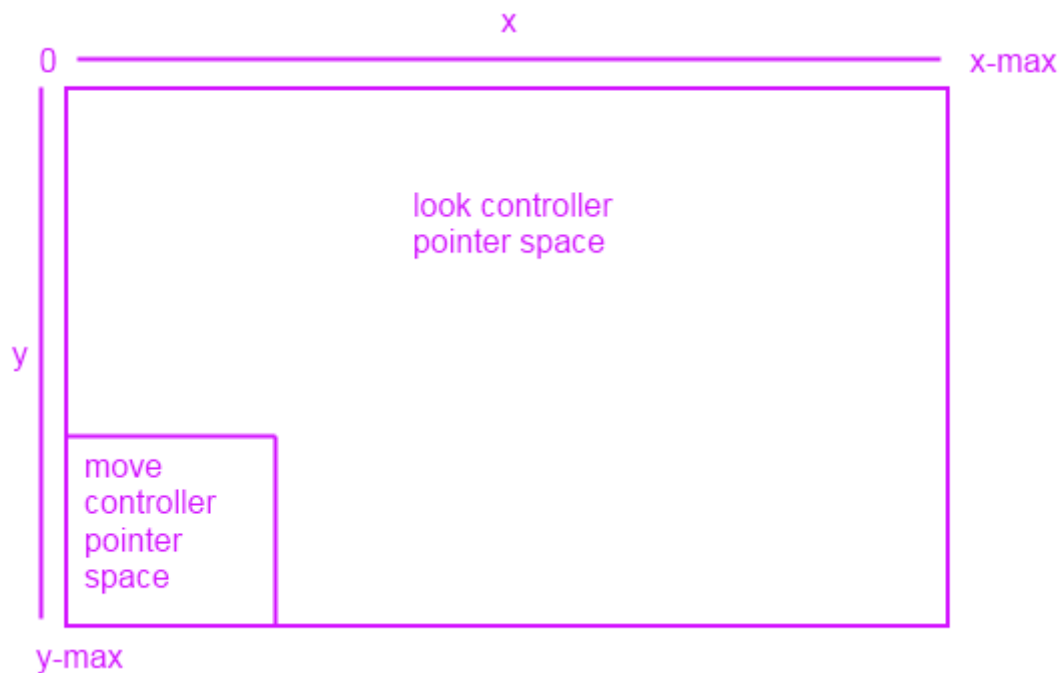
- Add basic move-look controls to your DirectX game for both mouse and keyboard, and touch screens.
- Implement a first-person camera used to navigate a 3D environment.

A note on touch control implementations

For touch controls, we implement two controllers: the move controller, which handles movement in the x-z plane relative to the camera's look point; and the look controller, which aims the camera's look point. Our move controller maps to the keyboard WASD buttons, and the look controller maps to the mouse. But for touch controls, we need to define a region of the screen that serves as the directional inputs, or the virtual WASD

buttons, with the remainder of the screen serving as the input space for the look controls.

Our screen looks like this.



When you move the touch pointer (not the mouse!) in the lower left of the screen, any movement upwards will make the camera move forward. Any movement downwards will make the camera move backwards. The same holds for left and right movement inside the move controller's pointer space. Outside of that space, and it becomes a look controller -- you just touch or drag the camera to where you'd like it to face.

Set up the basic input event infrastructure

First, we must create our control class that we use to handle input events from the mouse and keyboard, and update the camera perspective based on that input. Because we're implementing move-look controls, we call it **MoveLookController**.

C++

```
using namespace Windows::UI::Core;
using namespace Windows::System;
using namespace Windows::Foundation;
using namespace Windows::Devices::Input;
#include <DirectXMath.h>

// Methods to get input from the UI pointers
ref class MoveLookController
{
}; // class MoveLookController
```

Now, let's create a header that defines the state of the move-look controller and its first-person camera, plus the basic methods and event handlers that implement the controls and that update the state of the camera.

C++

```
#define ROTATION_GAIN 0.004f    // Sensitivity adjustment for the look
controller
#define MOVEMENT_GAIN 0.1f     // Sensitivity adjustment for the move
controller

ref class MoveLookController
{
private:
    // Properties of the controller object
    DirectX::XMFLOAT3 m_position;           // The position of the
controller
    float m_pitch, m_yaw;                 // Orientation euler angles in radians

    // Properties of the Move control
    bool m_moveInUse;                     // Specifies whether the move control is
in use
    uint32 m_movePointerID;               // Id of the pointer in this control
    DirectX::XMFLOAT2 m_moveFirstDown;     // Point where initial
contact occurred
    DirectX::XMFLOAT2 m_movePointerPosition; // Point where the move
pointer is currently located
    DirectX::XMFLOAT3 m_moveCommand;       // The net command from the
move control

    // Properties of the Look control
    bool m_lookInUse;                     // Specifies whether the look control is
in use
    uint32 m_lookPointerID;               // Id of the pointer in this control
    DirectX::XMFLOAT2 m_lookLastPoint;     // Last point (from last
frame)
    DirectX::XMFLOAT2 m_lookLastDelta;     // For smoothing

    bool m_forward, m_back;               // States for movement
    bool m_left, m_right;
    bool m_up, m_down;

public:

    // Methods to get input from the UI pointers
    void OnPointerPressed(
        _In_ Windows::UI::Core::CoreWindow^ sender,
        _In_ Windows::UI::Core::PointerEventArgs^ args
    );

    void OnPointerMoved(
        _In_ Windows::UI::Core::CoreWindow^ sender,
```

```

        _In_ Windows::UI::Core::PointerEventArgs^ args
    );

    void OnPointerReleased(
        _In_ Windows::UI::Core::CoreWindow^ sender,
        _In_ Windows::UI::Core::PointerEventArgs^ args
    );

    void OnKeyDown(
        _In_ Windows::UI::Core::CoreWindow^ sender,
        _In_ Windows::UI::Core::KeyEventArgs^ args
    );

    void OnKeyUp(
        _In_ Windows::UI::Core::CoreWindow^ sender,
        _In_ Windows::UI::Core::KeyEventArgs^ args
    );

    // Set up the Controls that this controller supports
    void Initialize( _In_ Windows::UI::Core::CoreWindow^ window );

    void Update( Windows::UI::Core::CoreWindow ^window );

internal:
    // Accessor to set position of controller
    void SetPosition( _In_ DirectX::XMFLOAT3 pos );

    // Accessor to set position of controller
    void SetOrientation( _In_ float pitch, _In_ float yaw );

    // Returns the position of the controller object
    DirectX::XMFLOAT3 get_Position();

    // Returns the point which the controller is facing
    DirectX::XMFLOAT3 get_LookPoint();

}; // class MoveLookController

```

Our code contains 4 groups of private fields. Let's review the purpose of each one.

First, we define some useful fields that hold our updated info about our camera view.

- **m_position** is the position of the camera (and therefore the viewplane) in the 3D scene, using scene coordinates.
- **m_pitch** is the pitch of the camera, or its up-down rotation around the viewplane's x-axis, in radians.
- **m_yaw** is the yaw of the camera, or its left-right rotation around the viewplane's y-axis, in radians.

Now, let's define the fields that we use to store info about the status and position of our controllers. First, we'll define the fields we need for our touch-based move controller. (There's nothing special needed for the keyboard implementation of the move controller. We just read keyboard events with specific handlers.)

- **m_moveInUse** indicates whether the move controller is in use.
- **m_movePointerID** is the unique ID for the current move pointer. We use it to differentiate between the look pointer and the move pointer when we check the pointer ID value.
- **m_moveFirstDown** is the point on the screen where the player first touched the move controller pointer area. We use this value later to set a dead zone to keep tiny movements from jittering the view.
- **m_movePointerPosition** is the point on the screen the player has currently moved the pointer to. We use it to determine what direction the player wanted to move by examining it relative to **m_moveFirstDown**.
- **m_moveCommand** is the final computed command for the move controller: up (forward), down (back), left, or right.

Now, we define the fields we use for our look controller, both the mouse and touch implementations.

- **m_lookInUse** indicates whether the look control is in use.
- **m_lookPointerID** is the unique ID for the current look pointer. We use it to differentiate between the look pointer and the move pointer when we check the pointer ID value.
- **m_lookLastPoint** is the last point, in scene coordinates, that was captured in the previous frame.
- **m_lookLastDelta** is the computed difference between the current **m_position** and **m_lookLastPoint**.

Finally, we define 6 Boolean values for the 6 degrees of movement, which we use to indicate the current state of each directional move action (on or off):

- **m_forward**, **m_back**, **m_left**, **m_right**, **m_up** and **m_down**.

We use the 6 event handlers to capture the input data we use to update the state of our controllers:

- **OnPointerPressed**. The player pressed the left mouse button with the pointer in our game screen, or touched the screen.
- **OnPointerMoved**. The player moved the mouse with the pointer in our game screen, or dragged the touch pointer on the screen.

- **OnPointerReleased.** The player released the left mouse button with the pointer in our game screen, or stopped touching the screen.
- **OnKeyDown.** The player pressed a key.
- **OnKeyUp.** The player released a key.

And finally, we use these methods and properties to initialize, access, and update the controllers' state info.

- **Initialize.** Our app calls this event handler to initialize the controls and attach them to the [CoreWindow](#) object that describes our display window.
- **SetPosition.** Our app calls this method to set the (x, y, and z) coordinates of our controls in the scene space.
- **SetOrientation.** Our app calls this method to set the pitch and yaw of the camera.
- **get_Position.** Our app accesses this property to get the current position of the camera in the scene space. You use this property as the method of communicating the current camera position to the app.
- **get_LookPoint.** Our app accesses this property to get the current point toward which the controller camera is facing.
- **Update.** Reads the state of the move and look controllers and updates the camera position. You continually call this method from the app's main loop to refresh the camera controller data and the camera position in the scene space.

Now, you have here all the components you need to implement your move-look controls. So, let's connect these pieces together.

Create the basic input events

The Windows Runtime event dispatcher provides 5 events we want instances of the **MoveLookController** class to handle:

- [PointerPressed](#)
- [PointerMoved](#)
- [PointerReleased](#)
- [KeyUp](#)
- [KeyDown](#)

These events are implemented on the [CoreWindow](#) type. We assume that you have a **CoreWindow** object to work with. If you don't know how to obtain one, see [How to set up your Universal Windows Platform \(UWP\) C++ app to display a DirectX view](#).

As these events fire while our app is running, the handlers update the controllers' state info defined in our private fields.

First, let's populate the mouse and touch pointer event handlers. In the first event handler, **OnPointerPressed()**, we get the x-y coordinates of the pointer from the **CoreWindow** that manages our display when the user clicks the mouse or touches the screen in the look controller region.

OnPointerPressed

C++

```
void MoveLookController::OnPointerPressed(
    _In_ CoreWindow^ sender,
    _In_ PointerEventArgs^ args)
{
    // Get the current pointer position.
    uint32 pointerID = args->CurrentPoint->PointerId;
    DirectX::XMFLOAT2 position = DirectX::XMFLOAT2( args->CurrentPoint-
    >Position.X, args->CurrentPoint->Position.Y );

    auto device = args->CurrentPoint->PointerDevice;
    auto deviceType = device->PointerDeviceType;
    if ( deviceType == PointerDeviceType::Mouse )
    {
        // Action, Jump, or Fire
    }

    // Check if this pointer is in the move control.
    // Change the values to percentages of the preferred screen resolution.
    // You can set the x value to <preferred resolution> * <percentage of
width>
    // for example, ( position.x < (screenResolution.x * 0.15) ).

    if (( position.x < 300 && position.y > 380 ) && ( deviceType !=
PointerDeviceType::Mouse ))
    {
        if ( !m_moveInUse ) // if no pointer is in this control yet
        {
            // Process a DPad touch down event.
            m_moveFirstDown = position; // Save the location
of the initial contact.
            m_movePointerPosition = position;
            m_movePointerID = pointerID; // Store the id of
the pointer using this control.
            m_moveInUse = TRUE;
        }
    }
    else // This pointer must be in the look control.
    {
        if ( !m_lookInUse ) // If no pointer is in this control yet...
        {
            m_lookLastPoint = position; // save the
point for later move
            m_lookPointerID = args->CurrentPoint->PointerId; // store the
id of pointer using this control
        }
    }
}
```

```

        m_lookLastDelta.x = m_lookLastDelta.y = 0;           // these are
    for smoothing
        m_lookInUse = TRUE;
    }
}
}

```

This event handler checks whether the pointer is not the mouse (for the purposes of this sample, which supports both mouse and touch) and if it is in the move controller area. If both criteria are true, it checks whether the pointer was just pressed, specifically, whether this click is unrelated to a previous move or look input, by testing if **m_moveInUse** is false. If so, the handler captures the point in the move controller area where the press happened and sets **m_moveInUse** to true, so that when this handler is called again, it won't overwrite the start position of the move controller input interaction. It also updates the move controller pointer ID to the current pointer's ID.

If the pointer is the mouse or if the touch pointer isn't in the move controller area, it must be in the look controller area. It sets **m_lookLastPoint** to the current position where the user pressed the mouse button or touched and pressed, resets the delta, and updates the look controller's pointer ID to the current pointer ID. It also sets the state of the look controller to active.

OnPointerMoved

C++

```

void MoveLookController::OnPointerMoved(
    _In_ CoreWindow ^sender,
    _In_ PointerEventArgs ^args)
{
    uint32 pointerID = args->CurrentPoint->PointerId;
    DirectX::XMFLOAT2 position = DirectX::XMFLOAT2(args->CurrentPoint->Position.X, args->CurrentPoint->Position.Y);

    // Decide which control this pointer is operating.
    if (pointerID == m_movePointerID)           // This is the move pointer.
    {
        // Move control
        m_movePointerPosition = position;       // Save the current
position.

    }
    else if (pointerID == m_lookPointerID)     // This is the look pointer.
    {
        // Look control

        DirectX::XMFLOAT2 pointerDelta;
        pointerDelta.x = position.x - m_lookLastPoint.x;           // How far
did pointer move
    }
}

```

```

    pointerDelta.y = position.y - m_lookLastPoint.y;

    DirectX::XMFLOAT2 rotationDelta;
    rotationDelta.x = pointerDelta.x * ROTATION_GAIN;    // Scale for
control sensitivity.
    rotationDelta.y = pointerDelta.y * ROTATION_GAIN;

    m_lookLastPoint = position;                          // Save for the next
time through.

                                                    // Update our
orientation based on the command.
    m_pitch -= rotationDelta.y;                          // Mouse y increases
down, but pitch increases up.
    m_yaw -= rotationDelta.x;                            // Yaw is defined as
CCW around the y-axis.

                                                    // Limit the pitch
to straight up or straight down.
    m_pitch = (float)__max(-DirectX::XM_PI / 2.0f, m_pitch);
    m_pitch = (float)__min(+DirectX::XM_PI / 2.0f, m_pitch);
}
}

```

The **OnPointerMoved** event handler fires whenever the pointer moves (in this case, if a touch screen pointer is being dragged, or if the mouse pointer is being moved while the left button is pressed). If the pointer ID is the same as the move controller pointer's ID, then it's the move pointer; otherwise, we check if it's the look controller that's the active pointer.

If it's the move controller, we just update the pointer position. We keep updating it as long as the **PointerMoved** event keeps firing, because we want to compare the final position with the first one we captured with the **OnPointerPressed** event handler.

If it's the look controller, things are a little more complicated. We need to calculate a new look point and center the camera on it, so we calculate the delta between the last look point and the current screen position, and then we multiply versus our scale factor, which we can tweak to make the look movements smaller or larger relative to the distance of the screen movement. Using that value, we calculate the pitch and the yaw.

Finally, we need to deactivate the move or look controller behaviors when the player stops moving the mouse or touching the screen. We use **OnPointerReleased**, which we call when **PointerReleased** is fired, to set **m_moveInUse** or **m_lookInUse** to FALSE and turn off the camera pan movement, and to zero out the pointer ID.

OnPointerReleased

C++

```

void MoveLookController::OnPointerReleased(
    _In_ CoreWindow ^sender,
    _In_ PointerEventArgs ^args)
{
    uint32 pointerID = args->CurrentPoint->PointerId;
    DirectX::XMFLOAT2 position = DirectX::XMFLOAT2( args->CurrentPoint-
        >Position.X, args->CurrentPoint->Position.Y );

    if ( pointerID == m_movePointerID )    // This was the move pointer.
    {
        m_moveInUse = FALSE;
        m_movePointerID = 0;
    }
    else if (pointerID == m_lookPointerID ) // This was the look pointer.
    {
        m_lookInUse = FALSE;
        m_lookPointerID = 0;
    }
}

```

So far, we handled all the touch screen events. Now, let's handle the key input events for a keyboard-based move controller.

OnKeyDown

C++

```

void MoveLookController::OnKeyDown(
    __in CoreWindow^ sender,
    __in KeyEventArgs^ args )
{
    Windows::System::VirtualKey Key;
    Key = args->VirtualKey;

    // Figure out the command from the keyboard.
    if ( Key == VirtualKey::W )    // Forward
        m_forward = true;
    if ( Key == VirtualKey::S )    // Back
        m_back = true;
    if ( Key == VirtualKey::A )    // Left
        m_left = true;
    if ( Key == VirtualKey::D )    // Right
        m_right = true;
}

```

As long as one of these keys is pressed, this event handler sets the corresponding directional move state to true.

OnKeyUp

C++

```
void MoveLookController::OnKeyUp(
    __in CoreWindow^ sender,
    __in KeyEventArgs^ args)
{
    Windows::System::VirtualKey Key;
    Key = args->VirtualKey;

    // Figure out the command from the keyboard.
    if ( Key == VirtualKey::W )    // forward
        m_forward = false;
    if ( Key == VirtualKey::S )    // back
        m_back = false;
    if ( Key == VirtualKey::A )    // left
        m_left = false;
    if ( Key == VirtualKey::D )    // right
        m_right = false;
}
```

And when the key is released, this event handler sets it back to false. When we call **Update**, it checks these directional move states, and move the camera accordingly. This is a bit simpler than the touch implementation!

Initialize the touch controls and the controller state

Let's hook up the events now, and initialize all the controller state fields.

Initialize

C++

```
void MoveLookController::Initialize( _In_ CoreWindow^ window )
{
    // Opt in to receive touch/mouse events.
    window->PointerPressed +=
        ref new TypedEventHandler<CoreWindow^, PointerEventArgs^>(this,
        &MoveLookController::OnPointerPressed);

    window->PointerMoved +=
        ref new TypedEventHandler<CoreWindow^, PointerEventArgs^>(this,
        &MoveLookController::OnPointerMoved);

    window->PointerReleased +=
        ref new TypedEventHandler<CoreWindow^, PointerEventArgs^>(this,
        &MoveLookController::OnPointerReleased);
}
```

```

        window->CharacterReceived +=
            ref new TypedEventHandler<CoreWindow^, CharacterReceivedEventArgs^>
            (this, &MoveLookController::OnCharacterReceived);

        window->KeyDown +=
            ref new TypedEventHandler<CoreWindow^, KeyEventArgs^>(this,
            &MoveLookController::OnKeyDown);

        window->KeyUp +=
            ref new TypedEventHandler<CoreWindow^, KeyEventArgs^>(this,
            &MoveLookController::OnKeyUp);

        // Initialize the state of the controller.
        m_moveInUse = FALSE;           // No pointer is in the Move
        control.
        m_movePointerID = 0;

        m_lookInUse = FALSE;           // No pointer is in the Look
        control.
        m_lookPointerID = 0;

        // Need to init this as it is reset every frame.
        m_moveCommand = DirectX::XMFLOAT3( 0.0f, 0.0f, 0.0f );

        SetOrientation( 0, 0 );         // Look straight ahead when the app
        starts.

    }

```

Initialize takes a reference to the app's **CoreWindow** instance as a parameter and registers the event handlers we developed to the appropriate events on that **CoreWindow**. It initializes the move and look pointer's IDs, sets the command vector for our touch screen move controller implementation to zero, and sets the camera looking straight ahead when the app starts.

Getting and setting the position and orientation of the camera

Let's define some methods to get and set the position of the camera with respect to the viewport.

```

C++

void MoveLookController::SetPosition( _In_ DirectX::XMFLOAT3 pos )
{
    m_position = pos;
}

// Accessor to set the position of the controller.

```



```

void MoveLookController::SetOrientation( _In_ float pitch, _In_ float yaw )
{
    m_pitch = pitch;
    m_yaw = yaw;
}

// Returns the position of the controller object.
DirectX::XMFLOAT3 MoveLookController::get_Position()
{
    return m_position;
}

// Returns the point at which the camera controller is facing.
DirectX::XMFLOAT3 MoveLookController::get_LookPoint()
{
    float y = sinf(m_pitch);           // Vertical
    float r = cosf(m_pitch);           // In the plane
    float z = r*cosf(m_yaw);           // Fwd-back
    float x = r*sinf(m_yaw);           // Left-right
    DirectX::XMFLOAT3 result(x,y,z);
    result.x += m_position.x;
    result.y += m_position.y;
    result.z += m_position.z;

    // Return m_position + DirectX::XMFLOAT3(x, y, z);
    return result;
}

```

Updating the controller state info

Now, we perform our calculations that convert the pointer coordinate info tracked in **m_movePointerPosition** into new coordinate information respective of our world coordinate system. Our app calls this method every time we refresh the main app loop. So, it is here that we compute the new look point position info we want to pass to the app for updating the view matrix before projection into the viewport.

C++

```

void MoveLookController::Update(CoreWindow ^window)
{
    // Check for input from the Move control.
    if (m_moveInUse)
    {
        DirectX::XMFLOAT2 pointerDelta(m_movePointerPosition);
        pointerDelta.x -= m_moveFirstDown.x;
        pointerDelta.y -= m_moveFirstDown.y;

        // Figure out the command from the touch-based virtual joystick.
        if (pointerDelta.x > 16.0f) // Leave 32 pixel-wide dead spot
            for being still.
    }
}

```

```

        m_moveCommand.x = 1.0f;
    else
        if (pointerDelta.x < -16.0f)
            m_moveCommand.x = -1.0f;

    if (pointerDelta.y > 16.0f)        // Joystick y is up, so change
sign.
        m_moveCommand.y = -1.0f;
    else
        if (pointerDelta.y < -16.0f)
            m_moveCommand.y = 1.0f;
}

// Poll our state bits that are set by the keyboard input events.
if (m_forward)
    m_moveCommand.y += 1.0f;
if (m_back)
    m_moveCommand.y -= 1.0f;

if (m_left)
    m_moveCommand.x -= 1.0f;
if (m_right)
    m_moveCommand.x += 1.0f;

if (m_up)
    m_moveCommand.z += 1.0f;
if (m_down)
    m_moveCommand.z -= 1.0f;

// Make sure that 45 degree cases are not faster.
DirectX::XMFLOAT3 command = m_moveCommand;
DirectX::XMVECTOR vector;
vector = DirectX::XMLoadFloat3(&command);

if (fabsf(command.x) > 0.1f || fabsf(command.y) > 0.1f ||
fabsf(command.z) > 0.1f)
{
    vector = DirectX::XMVector3Normalize(vector);
    DirectX::XMStoreFloat3(&command, vector);
}

// Rotate command to align with our direction (world coordinates).
DirectX::XMFLOAT3 wCommand;
wCommand.x = command.x*cosf(m_yaw) - command.y*sinf(m_yaw);
wCommand.y = command.x*sinf(m_yaw) + command.y*cosf(m_yaw);
wCommand.z = command.z;

// Scale for sensitivity adjustment.
wCommand.x = wCommand.x * MOVEMENT_GAIN;
wCommand.y = wCommand.y * MOVEMENT_GAIN;
wCommand.z = wCommand.z * MOVEMENT_GAIN;

// Our velocity is based on the command.
// Also note that y is the up-down axis.

```

```

DirectX::XMFLOAT3 Velocity;
Velocity.x = -wCommand.x;
Velocity.z = wCommand.y;
Velocity.y = wCommand.z;

// Integrate
m_position.x += Velocity.x;
m_position.y += Velocity.y;
m_position.z += Velocity.z;

// Clear movement input accumulator for use during the next frame.
m_moveCommand = DirectX::XMFLOAT3(0.0f, 0.0f, 0.0f);

}

```

Because we don't want jittery movement when the player uses our touch-based move controller, we set a virtual dead zone around the pointer with a diameter of 32 pixels. We also add velocity, which is the command value plus a movement gain rate. (You can adjust this behavior to your liking, to slow down or speed up the rate of movement based on the distance the pointer moves in the move controller area.)

When we compute the velocity, we also translate the coordinates received from the move and look controllers into the movement of the actual look point we send to the method that computes our view matrix for the scene. First, we invert the x coordinate, because if we click-move or drag left or right with the look controller, the look point rotates in the opposite direction in the scene, as a camera might swing about its central axis. Then, we swap the y and z axes, because an up/down key press or touch drag motion (read as a y-axis behavior) on the move controller should translate into a camera action that moves the look point into or out of the screen (the z-axis).

The final position of the look point for the player is the last position plus the calculated velocity, and this is what is read by the renderer when it calls the **get_Position** method (most likely during the setup for each frame). After that, we reset the move command to zero.

Updating the view matrix with the new camera position

We can obtain a scene space coordinate that our camera is focused on, and which is updated whenever you tell your app to do so (every 60 seconds in the main app loop, for example). This pseudocode suggests the calling behavior you can implement:

C++

```
myMoveLookController->Update( m_window );

// Update the view matrix based on the camera position.
myFirstPersonCamera->SetViewParameters(
    myMoveLookController->get_Position(),      // Point we are
    at                                          // Point to
    myMoveLookController->get_LookPoint(),      // Point to
    look towards                               // Up-vector
    DirectX::XMFLOAT3( 0, 1, 0 )
);
```

Congratulations! You've implemented basic move-look controls for both touch screens and keyboard/mouse input touch controls in your game!

Relative mouse movement and CoreWindow

Article • 10/20/2022

In games, the mouse is a common control option that is familiar to many players, and is likewise essential to many genres of games, including first- and third-person shooters, and real-time strategy games. Here we discuss the implementation of relative mouse controls, which don't use the system cursor and don't return absolute screen coordinates; instead, they track the pixel delta between mouse movements.

Some apps, such as games, use the mouse as a more general input device. For example, a 3-D modeler might use mouse input to orient a 3-D object by simulating a virtual trackball; or a game might use the mouse to change the direction of the viewing camera via mouse-look controls.

In these scenarios, the app requires relative mouse data. Relative mouse values represent how far the mouse moved since the last frame, rather than the absolute x-y coordinate values within a window or screen. Also, apps often hide the mouse cursor since the position of the cursor with respect to the screen coordinates is not relevant when manipulating a 3-D object or scene.

When the user takes an action that moves the app into a relative 3-D object/scene manipulation mode, the app must:

- Ignore default mouse handling.
- Enable relative mouse handling.
- Hide the mouse cursor by setting it a null pointer (nullptr).

When the user takes an action that moves the app out of a relative 3-D object/scene manipulation mode, the app must:

- Enable default/absolute mouse handling.
- Turn off relative mouse handling.
- Set the mouse cursor to a non-null value (which makes it visible).

Note

With this pattern, the location of the absolute mouse cursor is preserved on entering the cursorless relative mode. The cursor re-appears in the same screen coordinate location as it was previous to enabling the relative mouse movement mode.